

Oracle Business Intelligence Publisher Performance Recommendations for Fusion Cloud Applications

An Oracle White Paper, 1st Edition

August 16, 2021 | Version 1.0
Copyright © 2021, Oracle and/or its affiliates
Confidential – Public

PURPOSE STATEMENT

This document covers performance topics and best practices for Oracle Business Intelligence Publisher (BIP) for Fusion Cloud Applications Release 21B and higher. Most of the recommendations are generic for Fusion Applications releases. Release specific topics will refer to exact version numbers.

Note: The document is intended for BI system integrators, BI report developers and administrators. It covers advanced performance tuning techniques in BIP and Oracle RDBMS. All recommendations must be carefully verified in a test environment before applied to a production instance.

DISCLAIMER

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle software license and service agreement, which has been executed and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

This document is for informational purposes only and is intended solely to assist you in planning for the implementation and upgrade of the product features described. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

Due to the nature of the product architecture, it may not be possible to safely include all features described in this document without risking significant destabilization of the code.

TABLE OF CONTENTS

Purpose Statement	2
Disclaimer	2
Introduction	5
BIP Report Concepts	5
Understanding BIP Report Execution	6
BIP Performance Monitoring and Diagnostics	7
BIP Audit Self-Service Monitoring	7
BIP Diagnostic Output Analysis	8
BIP Data Model Validation	8
BIP Generate SQL Monitor Report	9
BIP View Engine Log	9
Guidelines for Analyzing Reports Performance	12
General Recommendations and Guidelines	12
BIP Reports Consistent Performance Across FA PODs	12
BIP SQL Plans Stability	13
BIP Data Model Performance Recommendations	13
Multiple Data Sets in BIP Data Model	13
Data Model Sharing Across Multiple Reports	14
Enable SQL Pruning in BIP Reports	14
Parent-Child (Nested) Data Sets in BIP Data Model	14
Parameters Defined as WITH SELECT FROM DUAL in Master-Child SQL Data Sets	15
BIP Data Set SQL Performance Recommendations	15
Function Calls in WHERE Clause Predicates	16
Custom PL/SQL Calls in SELECT Clause	17
Scalar Sub-Queries with DISTINCT Modifier or ROWNUM Keyword	17
Scalar Sub-Queries with Factored Sub-Queries	18
Unnecessary Scalar Sub Query in SELECT List	19
Aggregate Functions in Correlated Scalar Sub-query	19
Long in IN-LISTS in SQL Clauses	20
Data Security Predicates Wrapped Inside Redundant Sub-Query	21
Too Many Join Conditions Between Factored WITH Sub Queries	21
Redundant Table in LEFT OUTER JOIN	22
Redundant Table in Circular Inner Joins with Two or More Tables	23
Redundant Correlated Sub-Queries Inner-Joined in the Main Query	23
Use of NVL Functions With BIND Variables as Filters	24
Long CASE Expressions in SELECT and WHERE Clauses	24
Columns Defined as Literal Constants in Sub-Query Joined Later in Main Query	25
Large Number of UNION (ALL) Query Blocks in a Single SQL	25
Almost Identical Query Blocks in UNION (ALL) and INTERSECT	26
Redundant Table if Columns Can Be Fetched from Other Tables	26
Use XMLTYPE Data Type Instead of CLOB	27
Too Many Physical Row Sources (Tables, Views) in FROM Clause	27
The Same Table Used as a Lookup and a Dimension	28
Redundant Table in a Sub-Query	29
Missing Joins in SQL	29
Missing Filters in SQL	30
Too Many Column Expressions in GROUP BY Clause	31
Redundant Bridge Table	31
Use LIKE in addition to SUBSTR Scenarios	32
Leading Wildcard on LIKE Will Disallow Index Usage	32
_TL Tables Should Have a Filter on Language Column	33
Sub-Queries Having Connect-by-Prior	33
Outer Joins to _VL Views	34
Order by on Multiple Table(s) Columns	35
Rewrite Aggregate Sub-Query in WHERE to Inline View in FROM Clause	35

Redundant Attributes in SELECT Clause	35
Deeply Nested Decode in SQL	36
Attributes That May Force Late Filtering in Execution Plan	37
Positional Correlation in OR'ed EXISTS Clauses	38
Use of Complex Fusion Database Views in BIP SQLs	39
Avoid Reusing OTBI Generated SQL in BIP Data Sets	40
Attributes with Mandatory UPPER Function in Filters/Joins	40
Attributes with Mandatory TRUNC Function in Filters/Joins	40
Custom VPD Security in BIP SQLs	40
Use of Extensible Attributes in Filters and Joins	41
Anonymous PLSQL Blocks in Factored WITH Sub-Query	41
Sub-optimal CONCATENATION in Execution Plans	41
Fast Formula in BI Publisher Extract Reports	42
Reports and Data Model Properties	42
Data Caching for Online Reports	42
SQL Query Timeout	43
BIP Data Generation Performance Recommendations	43
BIP Reports Template Size	43
Data Format and Data Generation Time	43
CLOB impact on Data Generation	44
Functional Area Performance Recommendations	44
Financial BIP Reports	44
BIP Memory Guard Settings in FA SaaS	46
BIP Performance Errors: Recommendations and Workarounds	46
ORA-10260: limit size (1048576) of the PGA heap set by event 10261 exceeded	47
ORA-01722: invalid number	47
ORA-0600 Caused by Query Plan Exceeding 64K Plan Lines	47
ORA-01489: result of string concatenation is too long	47
Conclusion	48

INTRODUCTION

With Oracle Business Intelligence Publisher (BIP) real time analytic reports, Web Services (WS) data extracts, Oracle Enterprise Scheduler (ESS) flexible schedules, and Oracle Business Intelligence Enterprise Edition (OBIEE) integration, business users have a powerful means for accessing, interpreting and analyzing real-time data in Oracle Fusion Cloud Applications (FA). Users can easily create sophisticated reports with various custom attributes, filters, join conditions, and analytic functions using every single construct offered by Oracle SQL and PL/SQL. Proper design of analytic reports and data integration extracts in BIP is required to ensuring maximum performance and scalability, and to minimum impact on Fusion Middleware and Database resources in FA.

The BIP reports performance can be affected by the following factors:

- functional requirements for complex calculations and aggregations over large real-time data
- complex Datamodel design, involving multiple datasets
- complex SQL logic design of its dataset(s)
- data security clauses, either explicitly coded in a SQL, or coming from VPD predicates
- customizations and extensibility attributes
- web services data extracts and their execution frequency
- complex data formatting resulting in longer data generation
- processed data volumes, especially for scheduled integration reports
- data shape or volume changes that affect optimizer cardinalities estimates
- changes, such as patches or upgrades, affecting physical data model
- combination of the mentioned items above.

This document will cover each topic in more detail, with examples and recommendations on how to improve the design and to achieve better performance. Many examples will refer to SQL coding options, database query patterns, and SQL query execution paths. The target audience is expected to know SQL basics and to be able to read and understand sample execution plans.

Note, that FA offers more BI tools in its technology stack:

- Oracle Transactional Business Intelligence (OTBI) for quick and convenient custom and ad-hoc BI analysis
- Oracle Business Intelligence Cloud Connector (BICC) for extracting data in CSV output and publishing to Oracle Universal Content Management (UCM) Server or Oracle Storage Service.

Carefully evaluate all offered BI tools and choose the right one to address your functional BI requirements. BI Publisher may not be the best tool for data extracts, as it has enforced memory guards. BICC does not have such guardrails, so it is the primary extraction tool in FA SaaS. Refer to OTBI and BICC Performance Recommendations White Paper 2679006.1 for more details on OTBI and BICC usage and recommendations.

BIP Report Concepts

The concept of reporting in BIP consists of two major building blocks: BIP data model for designing report logic, and a template for designing report layout. Note that BIP data model can be shared by more than one report. BIP has separate sets of parameters and properties for a data model and a report - they define BIP execution context. Both BIP reports and BIP data models are stored in BI Web catalog. BIP data model operates with data sets to define logical report design. It can have more than one data set to manipulate with the data as per report functional requirements. Most common data set types are SQL queries, but BIP also supports other data set types (ref. to the Figure 1-1). In addition to data sets you

can also define more items in data model such as List of Values (LOVs), event triggers, parameters, and other options. Some of them will be discussed later in this paper. Multiple data sets in data model are defined via relationships, for example one data set's output can serve as input to another, or two data sets can be defined in parent-child relationship having one-to-many data set executions within the same data model.

Figure 1-1: BI Publisher Data Model



Report template defines report layout and any additional calculations (ref. to the Figure 1-2). You can generate report layout from Report Editor, Data Model Editor or Template Builder (using BIP plugins for MS Word or Excel). Refer to BIP documentation for more details.

Figure 1-2: RTF template calculation example

Grp_Supplier					
Supplier	Supplier 1				
Address	1 Long Avenue				
Invoice Number	Invoice Date	Currency	GL Date	Entered Amount	Accounted Amount
Grp:Invoice1134922	01-Jan-2007	USD	01-Jan-2007	\$100.00	\$100.00End Invoice
Total for Supplier: Supplier 1				\$100.00	\$100.00
End Supplier					
Invoice Tax Summary					
Tax Code		Entered Amt		Accounted Amt	
Grp:TaxVAT 17.5%		\$100.00		\$100.00End Tax	
Report Total		\$100.00		\$100.00	

Extracting the data via SQLs in data sets, joining in single data model, doing calculations in its template layout and generating the final output in desired format. These phases add up to the total report execution time and impact report performance.

Understanding BIP Report Execution

FA architecture supports two WebLogic Server (WLS) managed BI clusters in its deployments, BI cluster (bi_cluster1) running on two VMs: FA1 and FA2, and BIP Cluster (bip_cluster1), running on optional dedicated BIP VM: OPT3. Most the reports are executed on WLS managed cluster bi_cluster1, where BIP balances the load its two nodes: bi_server1 and bi_serverHA. BIP cluster bip_cluster1 is reserved for Payroll critical priority reports, scheduled via Enterprise Scheduling Service (ESS) only. BIP reports can be executed interactively (online) and scheduled (offline), they can be executed directly in BI Publisher or invoked via Web Services using BIP provided APIs.

When you run a report, BI Publisher invokes its own processors to execute the logical flow to retrieve the data, transform the output, generate required format and deliver it to the end user:

- First, BIP processes data model, it executes each SQL based data set serially, one-by-one, and in the order defined by relationships in its data model. This phase is database bound.
- Next, it transforms the data as per defined report layout and then generates the output into the chosen output format. This phase is BI(P) cluster bound. By default, BIP generates data in XML format in addition to user's choice of PDF, HMTL, CSV, XLS, etc.
- Lastly, BIP delivers the requested output either in online report, or as an attachment to the end user. This may involve a “bursting” sql to split the output and deliver to multiple users based on criteria in the “bursting query”.

BIP PERFORMANCE MONITORING AND DIAGNOSTICS

This section provides the recommendations and guidelines for BIP self-service monitoring and performance optimizations.

BIP Audit Self-Service Monitoring

FA enabled BIP report audit in database, starting from release 19B. All executed reports are recorded into BIP Audit tables in each FA POD. The audit captures all runtime metrics such as timestamps, timing, row counts, database SQLIDs, SQL text, error messages, and other metrics that can be analyzed for any performance issues, usage anomalies and trends in FA environments.

FA customers can implement self-service BIP audit monitoring and track any regressions / performance via BIP Audit reports. Refer to <https://www.oracle.com/technetwork/middleware/bi-publisher/downloads/index.html> for more details on template audit reports.

The example below shows a sample histogram report by report stored catalog path. From a quick glance, the key metrics of the number of errors, the total runs, and the runtime histogram can be used to prioritize poorly running reports for internal analysis:

Figure 2-1: BIP Audit Report example

BIP Audit - Histogram by Report Name																				
Report Name	# of Run	# of Unique Users	# of Errors	# of Cache Hit	0 to 1s	1 to 3s	3 to 5s	5 to 10s	10 to 20s	20 to 30s	30 to 60s	60 to 120s	2 to 5m	5 to 10m	10 to 30m	30m to 1hr	1 to 2hr	> 2hr	Max Time (s)	Avg Time (s)
/Custom/Customer Relationship Management/CRM_Develop/getFndLookupsReport.xdo	12336	1			12220	101	12	3											7.99	0.23
/Custom/Customer Relationship Management/Sales/CustomerConnect/GetCpys/CampaignRep.xdo	11124	1			1424	9595	78	19	2	4	1	1							115.81	1.29
/Custom/Customer Relationship Management/Sales/CustomerConnect/ProCpysRep.xdo	9100	1			8847	217	32	2	2										16.72	0.34
/Custom/Project Accounting/IT Reports/Reports/RA_OTL_CHECK_OTR.xdo	8629	1			8759	53	15	2											6.56	0.19
/Custom/Human Capital Management/HR/Recruiting/HR_Extract_Results.xdo	8047	1			7901	113	29	3	1										11.34	0.29
/Enterprise Contracts/Contract Terms Printing/ContractTermsHTMLPreview.xdo	6989	218	3		6919	48	20	2											5.44	0.35
/Enterprise Contracts/Transformation Templates/GenerateHTMLPreview.xdo	6983	218			6953	14	15	1											5.22	0.08
/Custom/Customer Relationship Management/Territories/SCA View Report.xdo	6673	1			6134	508	23	5	1		2								59.81	0.47
/Custom/Procurement/Custom Report/Tungsten_Details_Report.xdo	5242	1			5222	15	5												3.76	0.27
/Custom/Human Capital Management/Empl Comp History/Employment_History.xdo	4640	1588		1578	2268	1948	416	106			2								98.92	1.23
/Custom/Customer Relationship Management/Sales/CustomerConnect/SalesTeamCpysRep.xdo	4539	1			4442	85	9	3											8.36	0.33
/Financials/Payments/Disbursement/DisbursementAdviceFormats.xdo	3686	28			3660	19	7												4.87	0.04
/Custom/Customer Relationship Management/Sales/CloudDashboard/GetMyTerritoryAndTeamAccounts.xdo	3597	1			1754	1087	264	196	128	59	47	26	35	1					361.93	5.39
/Custom/Human Capital Management/Empl Comp History/Comp_History.xdo	3543	1669		724	1757	535	471	360	197	68	72	41	34	8					501.33	7.35
/Procurement/Self Service Procurement/RequisitionPDFGenerate.xdo	1788	362			1779	7	1	1											5.01	0.07
/Enterprise Contracts/Transformation Templates/DownloadClause.xdo	1775	181			1770	4	1												3.9	0.05
/Custom/Human Capital Management/Talent Review/Competency Gap Chart/Profile Gap Report.xdo	1403	300			998	301	85	19											6.78	1.06
/Custom/Customer Relationship Management/Sales/CustomerConnect/AccountSRRep.xdo	1146	1			1108	37		1											6.66	0.45
/Custom/Human Capital Management/HCM Reports/PAYROLL_DOCUMENT_OF_RECORD_EPY.xdo	1136	1			605	367	149	7	3		5								59.72	1.83
/Procurement/Purchasing/PurchaseDocumentEmailReport.xdo	1020	4			668	342	8	2											6.54	1.05
/Enterprise Contracts/Contract Terms Printing/ContractTermsLayout.xdo	917	15			910	4	2	1											5.69	0.38
/Enterprise Contracts/Transformation Templates/UploadClause.xdo	876	2			868	3	5												3.88	0.09

Review the following recommendations and guidelines for self-service BIP audit:

- BIP audit can be used to break down report runtime into query execution time in database vs. data transformation and generation time. You identify a report's time-consuming data set, its corresponding SQLIDs, retrieve SQL text, limiters, data set number of executions and row counts, as well as total report runtime, resultset rows and its size. Logical runtime analysis in BIP

audit always be the starting point, as it can provide the important insights about usage, performance metrics, report change history, etc.

- You can start with the reference templates to create a monitoring report, and then customize it with additional metrics for analysis as needed.
- BIP audit captures error messages of a report. Use it to analyze and resolve report design issues, report failures from memory guard limiters, or sub-optimal performance causing report timeout errors.
- The histogram can track the most frequent reports that are generating unnecessary workload on your environment and can help determine which reports require optimization due to its frequency.
- The BIP audit can track the change history for reports that all use the same list of values (LOVs). The SQLIDs of the data sets will not change if there are no changes made to the LOV's SQL. Any change to the SQL would result in the BIP audit to record a different SQLID. Such changes to the SQL can be tracked in the BIP Audit.

BIP Diagnostic Output Analysis

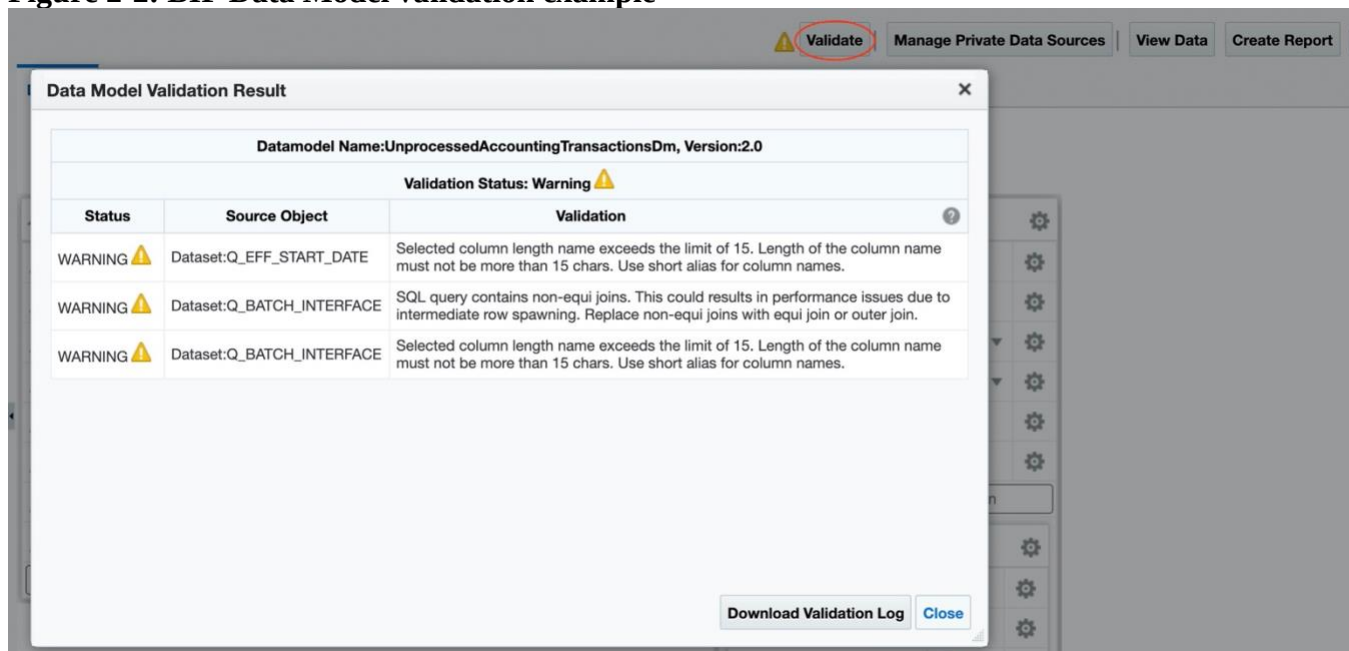
BI Publisher Enterprise provides several useful diagnostic tools, that can help to build more optimal reports during their design phase.

BIP Data Model Validation

When you design your data model and have all your data sets, including SQL data sets, ready, open or edit your data model and click 'Validate' button to generate the report with validation results for your data model. BIP will validate it against its internal set of the rules and provide the pointers to the datasets and describe the potential issues that require additional analysis and fixing as in the example below.

Important! New pattern design checks are added and enhanced for each BIP release. You are encouraged to periodically validate your existing critical reports, as well as newly created ones, against the updated rules.

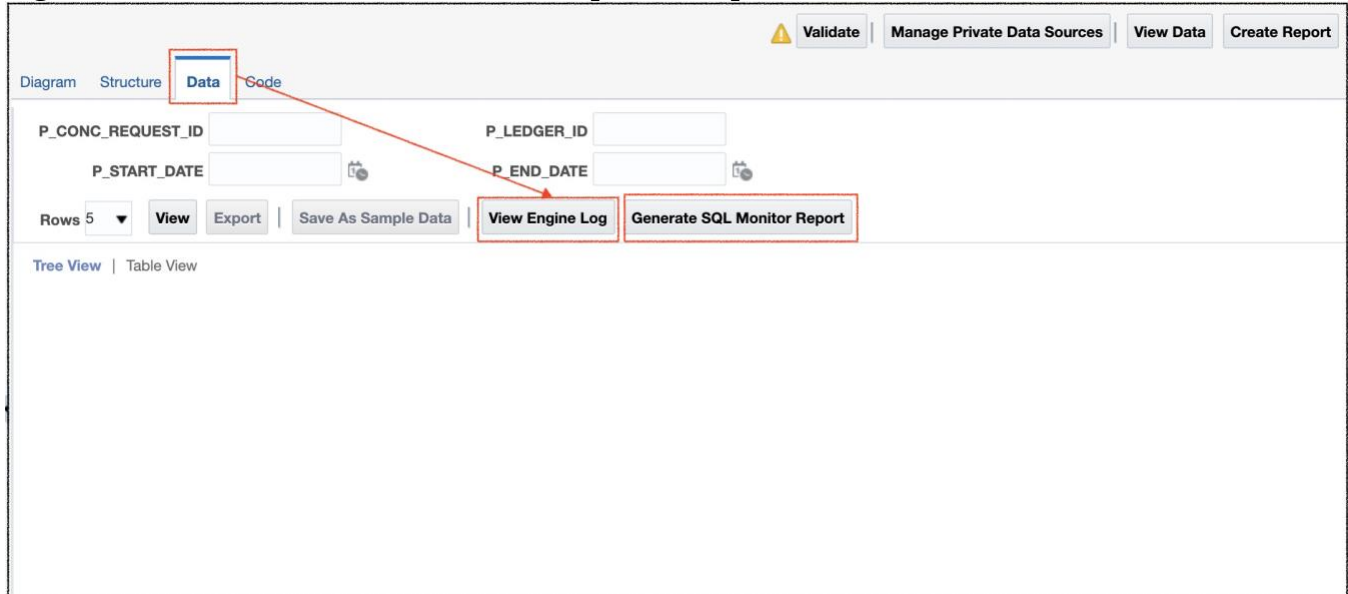
Figure 2-2: BIP Data Model validation example



BIP Generate SQL Monitor Report

You have an option to generate an explain plan for your data set directly in Query Builder by clicking ‘Generate Explain Plan’ or navigate to ‘Data’ tab and click ‘Generate SQL Monitor Report’ for all data sets. Refer to the Figure 2-3 below. In both cases you will get a generated explain plan with estimated costs, estimated join cardinalities and estimated run time.

Figure 2-3: BIP Generate SQL Monitor Report example



BIP View Engine Log

Refer to the Figure 2-3 above how to navigate and generate BIP ‘View Engine Log’ report. BIP generates very detailed output, showing the execution context with all data sets details. The most important diagnostic summary is shown below.

```
Start Datamodel diagnostic details..
=====
Data processor Id: Dp{1785400380}
Report absolute path: /Custom/Financials/Accounting/AccountingTrans.xdo
Datamodel name: _Custom_Financials_Data_Models_AccountingTranssDm_xdm
User name: faadmin
JDBC Session Module: BIPublisher_AccountingTrans
JDBC Session Action: Dp{1785400380}
Processing mode: online
Calling module: online
Enable Audit Events: true
Enable SQLID generation: true
StringList: 2
Cache SQL Rowset: false
Data Chunk Cache Size: 5242880
Optimize LOB Readers: true
Allow DB Modifications: false
Is DM Customized: true
Output format: 1
Auto db Fetchsize enabled: true
Default db Fetchsize : 20
SQL Query Timeout : 1800
```

```

Online SQL Query Timeout : 500
Offline SQL Query Timeout : 3600
XML data size limit: 524288000
XML Sample data size limit: 1048576
Processed with Scalable mode enabled: true
Track xml size enabled: true
SQL Session Trace enabled: false
SQL Session Trace File:
SQL Query Pruning enabled: false
Is Layout Elements passed: false
Layout Elements size: 0
Webservice timeout: 1800
Debug Mode: false
Estimate XML DataSize: false
Generate Output XML: true
DataSet Name:report
    Number of elements:8
    Total Query Execution time (ms):0
DataSet Name:Parameters
    Number of elements:4
    Total Query Execution time (ms):0
DataSet Name:Triggers
    Number of elements:2
    Total Query Execution time (ms):93
DataSet Name:Q_EFF_START_DATE
DataSet Type:sqlQuery-oracle
Data Source Name:ApplicationDB_FSCM
Is DataSet Execution Skipped:false
DataSet DB fetch size:100
DataSet Query Timeout:500
Total Number Of Times Query Executed:1
Total Number Of Rows Processed:1
Sql Action Id:Dp{1785400380}Sql{5207efd5}
Sql Session Details:SQLID: {aykj34hpvn08} SQL_HASH: {733827080}
SQL_EXEC_ID:{100663296}
    Number of elements:1
Total Query Execution time (ms):141
DataSet Name:Q_BATCH_INTERFACE
DataSet Type:sqlQuery-oracle
Data Source Name:ApplicationDB_FSCM
Is DataSet Execution Skipped:false
DataSet DB fetch size:50
DataSet Query Timeout:500
Total Number Of Times Query Executed:1
Total Number Of Rows Processed:5
Sql Action Id:Dp{1785400380}Sql{9c275aa4}
Sql Session Details: SQLID:{cx7n6h4a37s8q} SQL_HASH:{338944278}
SQL_EXEC_ID:{100663296}
    Number of elements:13
    Total Query Execution time (ms):163

-----
Total Number of Datasets: 5
Total Number of Elements: 28
Total Parse Time (ms): 1
Total Datamodel Validation Time: 98.000 mseconds
Total Xplan Generation Time: .000 mseconds
Total Data Generation Time: 522.000 mseconds
Total XML Data Size Generated: 3.22 KB

```

Highlighted in bold are the important fields and metrics to check for:

- **Data Processor ID** uniquely identifies all events for a single report execution.
- **Report absolute path** is a full catalog path for executed report or its datamodel.
- **Datamodel Name** is generated catalog path with '/' and spaces substituted with '_'. The same value gets stamped into each generated data set's SQL for tracking purposes.
- **User name** provide the user ID that execute the report.
- **Processing mode** can be online or scheduled, depending how the report was invoked.
- **Online SQL Query Timeout** shows the important guardrail for the query limiter. If the query takes longer than 500 sec during report online execution, BIP will terminate it and report the corresponding error message to the end user. This is a hard limit, as BIP uses BI managed server threads and has to release all threads under 600 sec.
- **Offline SQL Query Timeout** controls a SQL timeout for scheduled execution mode. The scheduled timeout limit is more relaxed as it does not involve any managed BI server threads.
Important! Oracle does not recommend changing this property and keep it enforced at the system level. Customized timeout values could result in heavy SQLs consuming more database resources and causing impact on the overall FA environment. Report developers should explore all the options to optimize their report SQLs and ensure them completing within enforced system limits.
- **XML data size limit** is another important guardrail governing the XML size of each BIP report generated XML output.
- **Dataset Name** block shows the runtime statistics for each and every data set in your data model.
- **Dataset Type** shows each dataset type, for example sqlQuery-oracle.
- **Dataset Query Timeout** reflects the same timeout as online limit, since the report was executed online.
- **Total Number Of Times Query Executed** tracks number of dataset SQL executions. It's a very important metric that could help to pinpoint design issues with parent-child data sets. Refer to the corresponding section below for more details.
- **Total Number Of Rows Processed** tracks total number of rows created by all the executions of the data set.
- **SQL Session Details:** contains Oracle Database generated SQLID for your query. If your datamodel SQL does not take variable number of BIND values passed, the SQL text remain unchanged, so SQLID can be used for tracking report changes in BIP Audit reports for any potential regressions.
- **Total Query Execution time (ms)** tracks the overall dataset runtime across all its executions, reported in milliseconds.
- **Total Data Generation time** (in milliseconds) reports the time spent in template processing. It could add to the total report runtime if you design very complex templates or choose more complex data generation formats (PDF or Excel).
- **Total Number of Datasets** opens the summary section for all elements, all datasets executions and provides the aggregated summary for your data model.

BIP uses the same format for logging each report execution in its 'bipublisher.log' files and records every single metric in its BIP audit tables for ease of monitoring.

Important! The data model execution summary does not provide the statistics for post data model data generation. The 'Data Generation Time' does not correspond to the time to generate XML output for your reports. BIP uses its internal tags start-transform / end-transform for online and start-scheduled-transform/end-scheduled-transform for scheduled report executions to log its data generation activities. The reported timestamps can be used for finding out the time for report data generation.

GUIDELINES FOR ANALYZING REPORTS PERFORMANCE

General Recommendations and Guidelines

When analyzing reports performance, use the following guidelines:

1. Use BIP audit to generate and review reports runtime histogram but limit to most recent date range (max 30 days). Running the audit for an extended range could result in longer report execution, as it performs complex aggregation for all entries. In very busy systems with many report executions, you may want to schedule such reports instead of trying them online.
2. The audit histogram report can help you to pinpoint several categories of reports for your analysis:
 - a. Too frequently run reports that may not need to be executed on such large scale. Reducing the execution frequency without compromising functional requirements would help to reduce the workload on your BIP cluster.
 - b. Reports executed by multiple users with high runtime histogram buckets or high maximum run time. These reports may need to be prioritized for your immediate attention as they could manifest into bigger issues and impact multiple end users.
 - c. Reports generating too high output need to be validated for the functional correctness. Pay special attention to those reports that reach the maximum report output limit. Most likely they produce functionally incomplete data (as the output would be truncated) and require immediate attention.
 - d. Any critical business reports, as part of FA business flows, such as Payroll or Order processing should be periodically checked to ensure their histogram being optional.
3. For each identified report generate its runtime history to track for its run time, extract rows, as well as SQLIDs. If you spot the SQLIDs changes over time, most likely report definitions have been updated and the report updates could be contributors to variable report performance.
4. Employ BIP Datamodel Validation to check the identified reports for any outstanding validation patterns and BIP SQL Monitor to review SQL explain plans.
5. Carefully review the next sections for most common sub-optimal BIP report design patterns and recommendations to add their sub-optimal performance.

BIP Reports Consistent Performance Across FA PODs

FA customers may have more than one development and test PODs for creating and testing BI reports before promoting them into Production environments. While DEV and TEST PODs are as close to PROD, there still could be few factors affecting the same BI reports, the same SQLs with the same SQLIDs, showing different performance:

1. PROD usually has more powerful configuration to accommodate for higher workload, better performance, and more scalability, while DEV and TEST may have less compute power.
2. DEV and TEST may not have such high and wide workload compared to PROD.
3. DEV and TEST may have comparable but not the same data volumes and most importantly, the exact data shapes. The latter could have the biggest impact on optimizer calculating the costs and potentially choosing different execution plans, resulting in varying performance between DEV/TEST and PROD.

A careful report design to avoid sub-optimal patterns, strong functional knowledge of the data model, effective use of filters and supporting indexes, well optimized SQL with all possible redundant logic and constructs not in the query, careful benchmarking, are all critical factors to ensure your reports consistent performance across environments for more end users and for larger data volumes.

BIP SQL Plans Stability

Database optimizer estimates costs for join cardinalities and generates the final execution plan with the lowest estimated cost for a SQL. There are many factors that could affect the cost estimates. A few of them are database version, database statistics, changes in data shapes, plan stability options such as SQL profiles, use of hints.

Review the following recommendations below:

- The best guidance for ensuring consistent BIP performance is to design simplified and well optimized SQLs. That would allow the optimizer to correctly calculate join cardinalities and choose the best execution plan.
- The use of database hints may be justified in very limited cases to force the optimizer to correct the execution plans, when you use complex calculations or apply functions that affect cardinality estimates. Database hints limit the optimizer in its choice of various optimization features; their misuse may have very negative impact to SQL performance.
- SQL plans and SQL profiles have very limited applications for plan stability too. Pinned plan hash values may not work for all data shapes. They will have no affect if you design BIP reports with variable number of bind variables resulting in SQLID changes for the same report. They could be considered only as short-term workarounds until the report SQL get redesigned and optimized.

BIP DATA MODEL PERFORMANCE RECOMMENDATIONS

BI Publisher Data Model design requires clear understanding of the functional requirements, careful planning, and most importantly proper logical design to ensure the best performance. This chapter focuses on the critical topics related to data model design in FA SaaS environments.

Multiple Data Sets in BIP Data Model

Keep your data model clean. Avoid creating too many data sets, if possible, consolidate into fewer or eliminate any unused data sets from your data model. BI Publisher executes data sets serially one-by-one, and it could result in very high accumulated run time in complex data models with too many data sets even with well-tuned individual data set SOLs.

Refer to the example below of the redundant data set Q_TAX, linked in parent-child relationship with Q_JELINES, unnecessarily executed 15,224 times, once for every single row produced by Q_JELINES.

Figure 3-1: Redundant Data Sets Example

[illegible]

Carefully inspect your logic and remove any unused Data Sets from your Data Model.

Data Model Sharing Across Multiple Reports

BI Publisher allows you to use a single data model for more than one report. This convenient feature can be used effectively, for example in more than one report that use different templates. However, data model sharing across multiple reports that use different data sets could lead to sub-optimal performance for all reports. When you execute a report, BIP data processor executes all data sets in the data model, including any redundant or unused data sets in the report, resulting in longer run time and slower performance.

Enable SQL Pruning in BIP Reports

Depending on Datamodel design and sharing across multiple reports not all Data Set SQLs may be required for specific report execution instance. If a Datamodel property “Enable SQL Pruning” is turned on, BIP processor would not execute unused Data Set SQLs, saving database resources and reduce your report runtime.

To enable the property:

1. Open your Datamodel and click Edit → Properties.
2. Locate the property “Enable SQL Pruning” and set its value to “On”.
3. Save the changes.

Parent-Child (Nested) Data Sets in BIP Data Model

BI Publisher’s feature to join two or more Data Sets in master-detail (or parent-child) relationship comes with the cost of running child data sets for as many times as there are parent query’s number of rows.

When implementing such master-detail logic in your reports consider the following:

1. Combine the parent and child SQL queries into a single data set, if they are not significantly different and especially if they join same or most of the same tables. The example below shows the effective consolidation of parent-child data sets, originally having Q_GL_BALANCES parent and Q_GL_JE_LINES child data sets, with Q_GL_BALANCES producing 20K balances and invoking Q_GL_JE_LINES 20K times:

```
WITH
  ldgr as (SELECT ... FROM gl_ledger_set_assignments glsa,
                        gl_ledgers gl2 WHERE ...),
  ccid as (SELECT ... FROM gl_code_combinations WHERE ...),
  ety  as (SELECT ...),
  ac   as (SELECT ...)
SELECT segment1 ety, ety.description, ...,
FROM   ety,
       ac,
       ...
       SELECT segment1, segment2, ... sum(...), sum(...)
           over (partition by gl_cc.segment1,...), ...
FROM     gl_code_combinations gl_cc
        ,gl_balances gbl
        ,periods
        ,gl_ledgers LED
WHERE ...
UNION ALL
SELECT segment1, segment2, ... sum(...), sum(...)
```

```

FROM      gl_code_combinations gl_cc
          ,gl_je_lines gjl
          ,gl_je_headers gjh
          ,gl_ledgers gl
          ,gl_periods gp

WHERE ...
    ) detailed_data
GROUP BY ldgr.name, segment1, segment2, ... segment7 ...
    ) aggregated_data

WHERE ...

```

2. Avoid querying very large transactional tables across parent-child data sets. Such tables usually have very few indexes and may not be useful in coded joins or filters, resulting in expensive full table scans. Combining parent-child data sets would help to reduce the number of times they are accessed in your BIP reports.
3. If the use of parent-child data sets cannot be avoided, ensure your child dataset SQLs are well optimized to use effective filters picking available indexes, and the filters applied early in the generated execution plans.

Parameters Defined as WITH SELECT FROM DUAL in Master-Child SQL Data Sets

Factored WITH AS SELECT ... FROM DUAL, that define parameters in SQL Data Sets could result in redundant executions and performance overhead in Datamodels that use Master-Child relationships. The better design is to define parameters (or lists) using SELECT from DUAL in separate PARAM Data Sets and pass the parameters into the required Master or Child Data sets.

Refer to the two examples below, showing sub-optimal use of WITH as SELECT ... FROM DUAL for a single parameter 'param' and a list 'period_list':

```

WITH param AS
(SELECT
  TO_DATE('1/1/' ||
    TO_CHAR(ADD_MONTHS(:P_AS_OF_DATE, - 12), 'YYYY'), 'MM/DD/YYYY') startdate,
  TO_DATE('12/31/' ||
    TO_CHAR(ADD_MONTHS(:P_AS_OF_DATE, - 12), 'YYYY'), 'MM/DD/YYYY') enddate,
  TO_CHAR(ADD_MONTHS(:P_AS_OF_DATE, - 12), 'YYYY') year
FROM DUAL
), ...

WITH period_list AS
  (SELECT regexp_substr(:G_PERIOD_NAMES, '[^,]+', 1, level) period_name
  FROM DUAL
   CONNECT BY regexp_substr(:G_PERIOD_NAMES, '[^,]+', 1, level) IS NOT NULL
  ), ...

```

BIP DATA SET SQL PERFORMANCE RECOMMENDATIONS

Review the following basic guidelines for developing performing SQLs for use in BIP Datamodel for Fusion Applications:

1. Know FA physical data model, its physical database objects and their attributes for developing performing SQLs in BIP Datamodel. Review periodically [Oracle Customer Connect](#) for FA

“Data Lineage” guides, posted after every FA release to understand the data model, database objects, attributes, indexes and index types to employ for your SQL queries faster performance.

2. Keep your SQL logic simple and clean. Copy-n-paste SQL logic from another report and adapt it to use in another report may result in redundant query blocks, database objects, missing joins, unnecessary attributes, making your SQL more complex, leading to increasing SQL parse time and less optimal execution plan.
3. Beware of the hidden complexity coming from the use of database views or security predicates appended via VPDs. Explain plans feature in BIP allows you to evaluate the impact from both.
4. Design your SQL logic to scale up with volumes and test it thoroughly in your DEV and TEST environments before putting into Production.
5. Perform regular audit for your designed BIP reports to track for reports slowing down over time. They may require additional analysis and optimization to ensure performing consistently for increased volumes and concurrency.

The next sections discuss the most common sub-optimal SQL design patterns observed in BIP reports in FA SaaS along with impact analysis and recommendations.

Function Calls in WHERE Clause Predicates

Expressions in WHERE clauses employ filter and join expressions to constrain rows from data sources (tables, views, inline views). When the estimated number of rows after filter/join operation using indexed attributes is low, the optimizer will choose an index to directly access the required data blocks. Applying SQL or PL/SQL functions to columns in filter/join expressions would disable index usage and revert table access to full table scan.

Alternatively, if there is a function-based index (FBI) defined, but filter or join expressions do not use the exact function in the join predicates, the optimizer will choose less optimal full table scan as well. For example, PERSON_NUMBER column in the table PER_ALL_PEOPLE_F is indexed via function-based index (FBI) using UPPER(PERSON_NUMBER). The appropriate use for PERSON_NUMBER in WHERE clause will be UPPER(PERSON_NUMBER). Refer to two separate sections on the recommended use of UPPER and TRUNC in WHERE clauses.

Consider the following example:

```
SELECT * FROM XLA_AE_HEADERS WHERE AE_HEADER_ID = '1511989';
```

There is a unique index defined on the primary key column AE_HEADER_ID, that makes access to a desired data very fast and efficient via INDEX RANGE SCAN operation:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	215	4 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID BATCHED	xla_ae_headers	1	215	4 (0)	00:00:01
* 2	INDEX RANGE SCAN	XLA_AE_HEADERS_U1	1		3 (0)	00:00:01

Predicate Information (identified by operation id):

```
2 - access("AE_HEADER_ID"=1511989)
```

When the function TO_CHAR() is used on AE_HEADER_ID column, the plan changes to below:

```
SELECT * FROM XLA_AE_HEADERS WHERE TO_CHAR(AE_HEADER_ID) = '1511989';
```

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		1	215	5788	(1)	00:00:01
* 1	TABLE ACCESS FULL	xla_ae_headers	1	215	5788	(1)	00:00:01

Predicate Information (identified by operation id):

1 - filter(TO_CHAR("AE_HEADER_ID")='1511989')

There may be valid cases where SQL or PL/SQL functions in WHERE clause are required by functional design. For example, aggregate functions in WHERE clauses (MAX, MIN, COUNT, AVG, etc.) may be needed for the functional logic. In such cases make sure you have effective filters on other columns with supporting indexes to ensure your reports optimal performance.

Custom PL/SQL Calls in SELECT Clause

Use of PL/SQL functions in SELECT clauses could lead to slower SQL performance. The optimizer has no methods to merge the code from PL/SQL functions into the main SQL query, so the functions are called as many times as the number of rows fetched by the main query. The PLSQL complexity multiplied by a number of its executions could become a major performance factor, often overlooked in SQL optimization.

Refer to an example below, showing the use of such function call, with another SELECT wrapped inside, with both the function and the SELECT invoked once for every fetched row from the main SELECT:

```
SELECT ...
  ff_user_tables_pkg.get_table_value(
    (SELECT l_data_grp.legislative_data_group_id
     FROM per_legislative_data_groups_vl l_data_grp
     WHERE l_data_grp.name = 'US'),
    'RETIREMENT_ELIGIBILITY',
    'AGE',
    to_char((months_between(sysdate, pp.date_of_birth) / 12)),
    trunc(sysdate)
  ) l_data_grp_pkg_call,
  ....
```

Make sure you review the use of such PL/SQL functions, and if their use required by the functional logic, apply effective filters to reduce the number of fetched rows and as the result number of the PL/SQL executions for the SQL.

Scalar Sub-Queries with DISTINCT Modifier or ROWNUM Keyword

Scalar sub-query is a SQL query block that is used as a column in a SELECT clause. It is called scalar because it must produce exactly one value (a scalar) to use in the main SELECT. If a sub-query is written in such a way that it returns more than one row ORA-01427 exception will be raised.

A typical usage pattern of scalar sub-queries is to deliver a single column value by joining a table inside the sub-query to an external table using a unique (primary key) column. That guarantees a single scalar value to be returned by such sub query.

If a scalar sub-query produces more than one row, either DISTINCT or ROWNUM may get (wrongly) used as workarounds to enforce a single scalar value. Both workarounds could result in incorrect functionality as well as cause performance overhead, as they get applied as the last step to potentially very large resultset produced by the sub-query.

```
SELECT
  mpm.level_member_id,
  archive.archive_id,
  mat.category_name,
  (
    SELECT DISTINCT category_name
    FROM msc_catalog_categories mccl
    WHERE mccl.category_id = mat.parent_category_id
      AND mccl.parent_category_id IS NOT NULL
  ) cgt_category_name,
  ...
```

Important! When you design your SQLs with correlated sub-queries in SELECT, do make sure they use index access path, as they get executed once for every single row. If a sub-query employs a full table scan due to the lack of indexes in join predicate, you end up with as many full table scans as the number of fetched rows in the main sub-queries. In that case you should rewrite it to use the join logic in WHERE clause instead of SELECT.

Scalar Sub-Queries with Factored Sub-Queries

As already discussed, scalar sub-queries are inline views, used in SELECT list. Factored sub-queries are defined using WITH clause and used in other parts of a SQL via unique aliases. Scalar sub queries are not un-nested by the optimizer; instead, they get executed once per every row produced by the main query. Factored sub queries are not un-nested either and very often they do not get materialized by default. As a result, performance of the query that contains such patterns will suffer from snowballing effect of progressively increased logical and/or physical reads.

Refer to the example below of such unnecessarily complex logic involving a scalar sub-query C2, using a factored WITH sub-query REQS called by another SELECT that uses another scalar sub-query REQUEST_ID, all within the parent scalar sub-query C2:

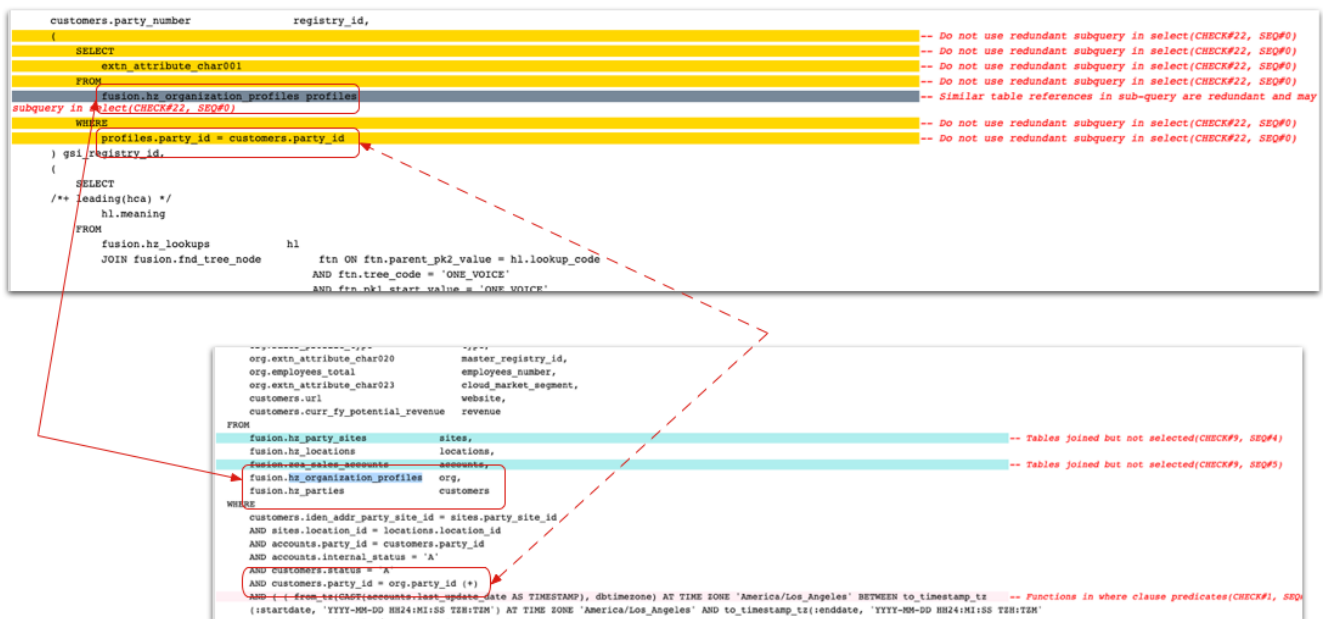
```
SELECT
  T1.C1 AS C1,
  (WITH REQS AS (
    SELECT RECORD_ID,
           REQUEST_ID
    FROM
      ALL_REQUESTS
    WHERE RECORD_ID = 100)
  SELECT
    (SELECT REQUEST_ID FROM REQS) AS REQUEST_ID
  FROM REQS
  WHERE ROWNUM < 2
  ) AS C2,
  T3.C3 AS C3
FROM ...
```

Closely inspect such complicated logic and explore the ways to rewrite to simplify the SQL.

Unnecessary Scalar Sub Query in SELECT List

Another sub-optimal scalar sub-queries usage pattern below involves a scalar sub-query in SELECT that may be eliminated, and the required column used directly in the SELECT list, if the same join is already present in the main query's FROM clause.

In the example below a correlated scalar sub query is used to define column GSI_REGISTRY_ID. Inside this sub query table HZ_ORGANIZATION_PROFILES is joined to another table, HZ_PARTIES, which is external to that sub query (hence correlated). This coding pattern effectively implements OUTER JOIN between HZ_ORGANIZATION_PROFILES and HZ_PARTIES. BUT both these tables are already present in the embracing query and there exists an OUTER JOIN condition between them (customers.party_id = org.party_id (+)). Therefore, correlated sub query is not needed there; instead extn_attribute_char001 column can be selected directly from HZ_ORGANIZATION_PROFILES table:



Aggregate Functions in Correlated Scalar Sub-query

Aggregate functions like MAX/MIN/COUNT in scalar subquery can be safely and efficiently be replaced by performing alternative re-writes. Refer to the example below:

--Sub-optimal pattern:

```
SELECT t1.c1,
       (SELECT MAX(t2.c2) as maxc2
        FROM t2
        WHERE t2.id=t1.id
       ) as c2
FROM t1;
```

--Optimal pattern:

```
SELECT t1.c1,
```

```

        t2alias.max_c2
FROM t1,
    (SELECT MAX(t2.c2) as max_c2
      T2.id as id
    FROM t2
    GROUP BY t2.id
    ) t2alias
WHERE t2alias.id=t1.id;

```

A scalar sub-query is executed for every row returned by the enclosing query block's row sources (here t1). It means the aggregated MAX will be executed as many times as the number of fetched rows in the main query, resulting in slower performance. Moving the aggregated MAX into FROM clause of the main query will aggregate the data once and speed up the overall query.

Note: If the number of rows returned by enclosing block (here t1) is moderate/low and the scalar sub-query is correlated on a selective indexed column then keeping the aggregate operation can be kept inside the correlated scalar sub-query.

Long in IN-LISTs in SQL Clauses

The option to pass more than one value to BIP SQL via bind parameters makes reports very flexible and versatile. However, BI Publisher does not limit the number of passed values, and it could generate a SQL with very large number of IN-LIST values, possibly hitting the internal Oracle limit of maximum 1,000 IN-LIST values (and get ORA-01795 maximum number of expressions in a list is 1000).

The Oracle optimizer converts IN-list expressions into Boolean OR-expressions. For instance, the following expression: COL_A IN (1, 2, 3) will be converted into COL_A = 1 OR COL_A = 2 OR COL_A = 3. Depending on how statistics are collected on a table and its columns, the optimizer may produce incorrect cardinality estimate for that IN-list expression. The more literals are in the IN-LIST, the larger deviation from the actual cardinality can be. The incorrect cardinality may cause the optimizer to select less join method. Refer to an example of long IN-LIST in CASE clause:

```

CASE WHEN d1.c40 NOT IN (
    'CALL',
    'CALL_IB',
    'DEMAND_GENERATION',
    'DEMO',
    'DEMO_IN_PERSON',
    'DISCOVERY',
    'DISCOVERY_PERSON',
    'EMAIL',
    'EMAIL_ELOQUA_ENGAGE',
    'EMAIL_ELOQUA_ENGCAMPAIGN',
    'EMAIL_IB',
    'EMAIL_OUTLOOK',
    'INPERSON_INTRO_MEETING',
    'INPERSON_MEETING',
    'INTRO_MEETING',
    'MANUAL_CALL',
    'MEETING',
    'RESEARCH',
    'SALES_CHAT',
    'SOCIAL_MEDIA',
    'TODO',

```

```
'WORKSHOPS '  
) THEN ...
```

Carefully design the parameters and lists of values that get passed to BIP reports to avoid generating long IN-LISTS in the SQLs. Do not create long checklists and an option to check all, as that would lead to generate such IN-clauses and affect performance for BI SQLs. To avoid passing 'all' and causing the performance with long IN-LISTS select 'NULL Value Passed' when you define the parameters for your datamodel. Then BIP would pass NULL to the bind variable instead of the long IN-LIST:

P_APPLICATION_ID: Type: Menu

Display Label: P_APPLICATION_ID

List of Values: LOV_APPLICATION_ID

Number of Values to Display in List: 100

Options: ☐ Multiple Selection

☒ Can select all

☒ NULL Value Passed ☐ All Values Passed

☐ Refresh other parameters on change

Data Security Predicates Wrapped Inside Redundant Sub-Query

Securing data in your reports requires very thorough analysis, and careful coding in your SQLs. Data Security Predicates (DSP) or security clauses, that you append to ensure secure reporting in your BIP SQLs, could have performance impact on your SQL execution. For example, if you choose to re-use DSPs from other BI SQLs that pull in View Object security clauses, you could end up with the pattern described below:

```
(SELECT ...  
  FROM T1,  
        (SELECT ...  
          FROM T2 ...  
          WHERE (DSP predicate1) OR (DSP predicate2)  
        )  
  WHERE ...
```

In this example table T2 got secured via two DSP predicates in a separate sub-query. As a part of overall optimization process Oracle optimizer performs recursive optimization of each inline view (sub query) before optimizing the main query. Such pattern could affect the query parsing time and possibly produce less efficient execution plan. Consider using more efficient pattern rewrite:

```
(SELECT ...  
  FROM T1,  
        T2, ...  
  WHERE (DSP predicate1) OR (DSP predicate2))  
  ...
```

Too Many Join Conditions Between Factored WITH Sub Queries

When you join two factored WITH sub-queries using too many join conditions, the optimizer may estimate low join cardinality for the respective tables and not choose the best execution plan. Refer to the example below, showing the sub-optimal generated SQL pattern as the result of having too many joins between two factored WITH sub-queries:

```
FROM SAWITH1 D1
  FULL OUTER JOIN SAWITH2 D2
    ON SYS_OP_MAP_NONNULL(D1.c3) = SYS_OP_MAP_NONNULL(D2.c3)
    AND SYS_OP_MAP_NONNULL(D1.c5) = SYS_OP_MAP_NONNULL(D2.c5)
    AND SYS_OP_MAP_NONNULL(D1.c12) = SYS_OP_MAP_NONNULL(D2.c12)
    AND SYS_OP_MAP_NONNULL(D1.c14) = SYS_OP_MAP_NONNULL(D2.c14)
    AND SYS_OP_MAP_NONNULL(D1.c11) = SYS_OP_MAP_NONNULL(D2.c11)
    AND SYS_OP_MAP_NONNULL(D1.c2) = SYS_OP_MAP_NONNULL(D2.c2)
    AND SYS_OP_MAP_NONNULL(D1.c13) = SYS_OP_MAP_NONNULL(D2.c13)
    AND SYS_OP_MAP_NONNULL(D1.c15) = SYS_OP_MAP_NONNULL(D2.c15)
    AND SYS_OP_MAP_NONNULL(D1.c9) = SYS_OP_MAP_NONNULL(D2.c9)
    AND SYS_OP_MAP_NONNULL(D1.c10) = SYS_OP_MAP_NONNULL(D2.c10)
    AND SYS_OP_MAP_NONNULL(D1.c8) = SYS_OP_MAP_NONNULL(D2.c8)
    AND SYS_OP_MAP_NONNULL(D1.c7) = SYS_OP_MAP_NONNULL(D2.c7)
    AND SYS_OP_MAP_NONNULL(D1.c18) = SYS_OP_MAP_NONNULL(D2.c18)
    AND SYS_OP_MAP_NONNULL(D1.c21) = SYS_OP_MAP_NONNULL(D2.c21)
    AND SYS_OP_MAP_NONNULL(D1.c4) = SYS_OP_MAP_NONNULL(D2.c4)
    AND SYS_OP_MAP_NONNULL(D1.c6) = SYS_OP_MAP_NONNULL(D2.c6)
    AND SYS_OP_MAP_NONNULL(D1.c22) = SYS_OP_MAP_NONNULL(D2.c22)
    AND SYS_OP_MAP_NONNULL(D1.c20) = SYS_OP_MAP_NONNULL(D2.c20)
    AND SYS_OP_MAP_NONNULL(D1.c16) = SYS_OP_MAP_NONNULL(D2.c16)
    AND SYS_OP_MAP_NONNULL(D1.c17) = SYS_OP_MAP_NONNULL(D2.c17)
    AND SYS_OP_MAP_NONNULL(D1.c19) = SYS_OP_MAP_NONNULL(D2.c19)
```

Carefully review all the joins in your SQL logic, and keep them to bare minimum to address the report functional requirements.

Redundant Table in LEFT OUTER JOIN

Some of Left Outer Joins (LOJ) may be redundant if you join two tables via LOJ but not select any attributes from the table on the 'optional' side of the LOJ. Review the example below:

```
SELECT
  b.book_title,
  b.author,
  b.language
FROM
  books b,
  book_languages l
WHERE
  b.language = l.language (+);
```

The table BOOK_LANGUAGES is on the *optional side* of the OUTER join and no columns from it are in SELECT clause. The query above is functionally equivalent (i.e., it produces the same exact result set) with both the table and LOJ removed:

```
SELECT
  b.book_title,
  b.author,
  b.language
```

```
FROM
    books b;
```

Complex BIP SQL queries with redundant OUTER joined tables may result in additional performance overhead during parse time as well as at run time. The optimizer may be unable to eliminate unused outer joined table as outer joins are usually harder to process than the ordinary inner joins. Make sure you carefully validate your functionality to evaluate the impact from removing such joins and tables in your reports.

Redundant Table in Circular Inner Joins with Two or More Tables

Some of very complex SQLs may result in three tables having circular joins via inner joins, as in the example below:

```
SELECT
    b.application_id,
    d.transaction_number,
    b.event_id,
    d.product_id
FROM
    xla_ae_headers b,
    xla_transaction_headers d,
    xla_events c
WHERE
    b.event_id = c.event_id
AND    b.event_id = d.event_id
AND    c.event_id = d.event_id;
```

1. The proper design for N circular tables uses N-1 joins among them. Optimizer may miscalculate join cardinalities with redundant joins leading to less optimal execution plans.
2. In the example above XLA_EVENTS (C) is potentially a redundant table, as it has no columns in SELECT clause:
 - a. The join between XLA_EVENTS (C) and XLA_TRANSACTION_HEADERS (D) is implied automatically due to transitive property of inner join operation.
 - b. The optimizer will unlikely remove table XLA_EVENTS. Having extra unused table (especially a large one) in the query will decrease performance due to additional join overhead.
 - c. Consider removing such object(s) if that would not cause functional impact on your SQL logic.

Redundant Correlated Sub-Queries Inner-Joined in the Main Query

The more complex case describes the pattern with the redundant correlated sub-query, where one query block is a superset of the second query block, and both are inner-joined later in the main query. Refer to the example below:

```
SELECT
    QB1.s_k_5000 c1,
    QB2.s_k_5000 c2
FROM
    (SELECT
```

```

biflexfieldeo.assignment_id AS s_k_5000,
biflexfieldeo.ass_attribute10,
(SELECT Substr(NAME, 1, 10)
FROM per_grades
WHERE (Substr(NAME,1,10)=biflexfieldeo.ass_attribute15 )) AS worker_grade
FROM per_all_assignments_m biflexfieldeo) QB1,
(SELECT biflexfieldeo.assignment_id AS s_k_5000,
NAME s_name,
biflexfieldeo.ass_attribute10
FROM per_all_assignments_m biflexfieldeo) QB2
WHERE QB1.worker_grade_ = QB2.s_name;

```

The first query block QB1 effectively is an OUTER join of PER_ALL_ASSIGNMENTS_M and PER_GRADES with worker_grade IS NULL wherever (Substr(NAME, 1, 10) != biflexfieldeo.ass_attribute15. The second query block QB2 is a sub-set of QB1 result set, where all rows that have NULLs in worker_grade are skipped. The thorough analysis of inline views QB1 and QB2 shows that the QB1 is a superset of QB2, with the extra rows having NULL in worker_grade column. When the two inline views are later joined on equi-join predicate, the final result set will have the rows from QB2 only. So, the entire query can be rewritten without an extra view QB1 and additional filters applied to QB2. The pattern complicates overall logic of the query and adds extra overhead due to redundant inline view with scalar sub query in its SELECT list.

Use of NVL Functions With BIND Variables as Filters

Avoid NVL functions on the filters that use bind variables. The following sub-optimal pattern could result in the Optimizer mis-calculating the cardinality for the filtered column:

```

AND (CProgramEO.CARD_PROG_NAME=NVL(:P_PROG_NAME, CProgramEO.CARD_PROG_NAME))

```

Instead, rewrite to use the following equivalent logic:

```

AND (:P_PROG_NAME IS NULL OR CProgramEO.CARD_PROG_NAME = :P_PROG_NAME)

```

Long CASE Expressions in SELECT and WHERE Clauses

Complex CASE expressions can affect query performance in multiple ways. CASE expressions found in SELECT list are CPU-intensive operations. CASE expressions in WHERE clause (filter and join predicates) may result in incorrect join cardinality estimate and miscalculating the costs. As a result, sub-optimal join method can be selected causing potentially long run time of the query, increasing probability of **ORA-01555** (snapshot too old) error and significant TEMP tablespace usage by the SQL. Carefully inspect your SQL for any large CASE clauses and explore the options to simplify the logic. Refer to the sub-optimal and rewritten CASE structures below:

```

-- Sub-optimal structure:

(CASE WHEN .. THEN
  (CASE WHEN .. THEN
    (CASE WHEN .. THEN
      (...)
    END)
  END)
ELSE ... END)

-- Rewritten optimal structure:

```

```
(CASE WHEN ... THEN
      WHEN ... THEN
      WHEN ... THEN
      ELSE ...
END)
```

Columns Defined as Literal Constants in Sub-Query Joined Later in Main Query

In the cases where you have literal constants, defined as columns in a single sub-query and later used in joins in the main query, consider passing them as direct filters instead. Refer to the example of such usage below:

```
(SELECT
  ...
  UPPER('HXT') AS DUMMY
  ...
) V1,
(SELECT
  T2.C2 AS C2
FROM T2
) V2
WHERE
  V1.DUMMY = V2.C2
...
```

The more efficient SQL design would be to pass it directly to the main query:

```
(SELECT
  ...
) V1,
(SELECT
  T2.C2 AS C2
FROM T2
) V2
WHERE
  V2.C2 = UPPER('HXT')
...
```

Large Number of UNION (ALL) Query Blocks in a Single SQL

A single SQL with multiple UNION (or UNION ALL) query blocks should be carefully analyzed to consolidate UNIONS into fewer sub-queries. The excessive use of UNIONS would be a strong indicator of logic redundancy and SQL complexity. It results in much larger SQL statement, which would take much longer to parse, and execute in the database, as each UNION query block by default is executed serially, one-by-one. Forcing `/*+ PARALLEL */` hint would not help to run multiple UNION branches at the same time. The hint `/*+ PQ_CONCURRENT_UNION */` could help with parallel UNION branch executions, but it could result in more DB workload, especially if the UNION branches have very expensive tables and no selective filters.

In the most cases the Union branches may be nearly identical with minor differences such as slightly different filters or different expressions on columns in SELECT lists, etc. Consider reworking multiple-UNION query into a much simpler shape by factoring out one or two common denominator WITH sub-queries and reusing them as per functional requirements. Such approach would result in much less

logical reads, lesser database impact and faster query performance. If the same WITH factored sub-query called more than once, Oracle would ‘materialize’ its resultset in a temporary segment and use it throughout the SQL.

Almost Identical Query Blocks in UNION (ALL) and INTERSECT

A SQL with structurally and semantically similar branches, joined via UNION(ALL) or INTERSECT operations can be optimized to consolidate almost identical branches into one branch or use WITH factored sub-query and then join the materialized result set as needed. Refer to the example below:

```
SELECT C1 AS O1, C2 AS O2 FROM T1 WHERE T1.C3 = 1
UNION
SELECT C1 AS O1, C2 AS O2 FROM T1 WHERE T1.C3 = 2;
```

A quick analysis of this query suggests that it can be rewritten as follows:

```
SELECT C1 AS O1, C2 AS O2 FROM T1 WHERE T1.C3 IN (1,2);
```

In the real-world Fusion Applications using BIP reporting, it is very important to carefully inspect the use of UNIONs and weigh very carefully the benefits from constructing factored WITH sub-queries that may produce large result sets and involve heavy I/O vs. few UNION branches if they can be executed with help of effective indexes and vice versa.

Redundant Table if Columns Can Be Fetched from Other Tables

Closer analysis of inner joins used in a SQL could help to identify potentially redundant tables if they have no additional joins to other tables. Refer to the example below:

```

SELECT
    DocumentsVL.URL                AS INV_URL_LINK ,
    DocumentsVL.Title              AS INV_URL_TITLE ,
    Invoices.INVOICE_ID            AS INV_URL_ID ,
    DocumentsVL.Description        AS INV_URL_DESC ,
    FndDocumentCategoriesEO.USER_NAME AS INV_URL_CAT_NAME ,
    Documents.CREATED_BY          AS INV_URL_CREATED_BY ,
    LookUps.Meaning                AS INV_URL_TYPE ,
    Documents.CREATION_DATE       AS INV_URL_CREATE_DATE
FROM
    FND_DOCUMENTS_VL DocumentsVL ,
    FND_ATTACHED_DOCUMENTS Documents ,
    AP_INVOICES_ALL Invoices ,
    fnd_lookups LookUps ,
    FND_DOCUMENT_CATEGORIES_VL FndDocumentCategoriesEO
WHERE
    Documents.ENTITY_NAME LIKE 'AP_INVOICES_ALL'
    AND Documents.DOCUMENT_ID = DocumentsVL.DOCUMENT_ID
    AND DocumentsVL.DATATYPE_CODE LIKE 'WEB_PAGE'
    AND LookUps.lookup_type = 'FND_DOCUMENT_DATATYPES'
    AND DocumentsVL.DATATYPE_CODE = LookUps.LOOKUP_CODE(+)
    AND Documents.category_name = FndDocumentCategoriesEO.category_name(+)
    AND Documents.PK1_VALUE = Invoices.Invoice_ID
    AND :INV_ID = Invoices.INVOICE_ID
ORDER BY Documents.LAST_UPDATE_DATE ASC

```

The table AP_INVOICES_ALL is not required, as it is INNER joined on the same column Invoice_ID to another table FND_ATTACHED_DOCUMENTS and there are no more joins to other tables. Invoice_ID could be filtered (INVOICE_ID= :INV_ID) and its values retrieved from the table FND_ATTACHED_DOCUMENTS.

Query optimizer has no knowledge of developer's intentions for adding the redundant table to a query so it likely would not be eliminate it during parsing/optimization.

Use XMLTYPE Data Type Instead of CLOB

Avoid using CLOB data type for storing XML data. The conversion of CLOB to XML on the fly using XMLTYPE function during report runtime could be very expensive. Instead, consider using XMLTYPE for storing XML documents in the database.

Too Many Physical Row Sources (Tables, Views) in FROM Clause

Having too many row sources in FROM clause may heavily affect your SQL performance:

- Query parse time is impacted negatively - the more tables are in the FROM clause the longer it takes to the optimizer to compute the best join order.
- Chances to find the best join order become significantly lower with more row sources. The optimizer tries different permutations of joins to find the order with minimal cost. The more tables are in the FROM clause the lower are chances to find the best (or *good enough*) join order.

- The number of join conditions between n tables should be at least (n-1), though usually SQLs have many more joins. With more join conditions there are worse chances to get good enough cardinality estimates.
- Heavy database views in the same query could increase the query complexity as well as the number of physical row sources.

The example below shows 22 tables and views joined in a single UNION branch:

```
FROM
  pjc_exp_items_all pje,
  pjf_exp_types_tl petv,
  pjc_cost_dist_lines_all prj_cost,
  hr_all_organization_units org,
  pjf_tasks_v ptv,
  ap_invoices_all ai,
  ap_invoice_distributions_all aida,
  gl_je_headers gjh,
  gl_je_lines gjl,
  gl_import_references gir,
  xla_ae_headers xah,
  xla_ae_lines xal,
  xla_events xe,
  xla_distribution_links xdl,
  fnd_id_flex_segments ffs,
  fnd_flex_values_vl ffv,
  xle_entity_profiles legal,
  pjf_txn_sources_tl pts,
  gl_ledgers gl,
  pjf_projects_all_tl prt,
  hr_organization_units hou,
  gl_code_combinations gcc_cost
WHERE ...
```

Carefully review every single table and view and keep the list of queried objects to bare minimum in your SQLs.

The Same Table Used as a Lookup and a Dimension

This sub-optimal pattern involves the same table used as a lookup in one sub-query, and as a dimension in another query, with both later joined on the same column later in the main query.

Refer to the example, describing such scenario below:

```
WITH
  SAWITH0 AS
    (SELECT...
      T25439.CODE AS c20,
      ...
    FROM ...
    DWH.DW_CODE_VALUES_LKP_TL T25439/*Dim_CODE_VALUES_LKP_TL_AP_Payment_Status*/,
    ...),
  SAWITH1 AS
    . (SELECT
      T25439.NAME AS c1,
      T25439.CODE AS c2
    FROM
```

```

DWH.DW_CODE_VALUES_LKP_TL T25439/*Dim_CODE_VALUES_LKP_TL_AP_Payment_Status*/
WHERE T25439.LANGUAGE = 'US'
AND T25439.CODE_TYPE = 'CHECK STATE'
),...
SELECT ...
FROM ((SAWITH0 D1 LEFT OUTER JOIN SAWITH1 D2 ON D1.c20 = D2.c2)
...

```

The same table `DW_CODE_VALUES_LKP_TL` is used twice, one time as a lookup and second time as a dimension and then these two references are joined to each other on the same column thus creating no new functional contents. Such redundancy may result in performance impact:

- Using table as a lookup generates one more factored sub query that requires extra steps in parsing and optimization, increasing the query parse time.
- Having redundant objects and structures may result in less optimal execution plan
- Extra scan/filter/join operations on the redundant object may increase query run time

Redundant Table in a Sub-Query

The same table may be considered redundant and eliminated from a SQL if it is used in a sub-query and the main query and joined in the main query WHERE clause. Consider the following example:

```

1 SELECT
2   expensetypes.name           expense_type_name,
3   expenses.expense_id        expense_id,
4   expenses.start_date        start_date,
5   expenses.receipt_amount    receipt_amount,
6   expenses.receipt_currency_code curr_code,
7   expenses.location          exp_location
8 FROM
9   exm_expenses              expenses,
10  exm_expense_types         expensetypes,
11  exm_lookup_values         exmlookup,
12  exm_expense_dists         exmexpensedists,
13  exm_expense_reports       exmexpensereports
14 WHERE
15   exmexpensereports.expense_report_id = expenses.expense_report_id
16   AND expenses.expense_type_id = expensetypes.expense_type_id
17   AND expenses.expense_id = exmexpensedists.expense_id
18   AND expenses.expense_category_code = exmlookup.lookup_code
19   AND ( exmexpensereports.parent_expense_report_id IS NULL )
20   AND expensetypes.name IN (
21     SELECT
22       expensetypes.name
23     FROM
24       exm_expenses              expenses,
25       exm_expense_types         expensetypes,
26       exm_lookup_values         exmlookup,
27       exm_expense_dists         exmexpensedists,
28       exm_expense_reports       exmexpensereports
29     WHERE
30       exmexpensereports.expense_report_id = expenses.expense_report_id
31       AND expenses.expense_type_id = expensetypes.expense_type_id
32       AND expenses.expense_id = exmexpensedists.expense_id

```

Annotations in the original image:

- Red boxes highlight the table lists in the main query (lines 9-13) and the sub-query (lines 24-28).
- Red arrows point from the sub-query table list to the main query table list, indicating redundancy.
- Red text annotations state: "Similar table references in sub-query are redundant and may be removed(CHECK#21, SEQ#0)" and "Similar table references in sub-query are redundant and may be removed(CHECK#21, SEQ#1)".

In this example `exm%` tables are used in the main query and in an inline query, joined via of IN-expression. The column extracted in the inline query, is also in the main query SELECT clause. The join conditions between these two tables are duplicated in both places. This logic can be simplified by consolidating both parts without having any functional impact on the SQL.

Such sub-optimal usage pattern with unnecessary redundancy could lead to longer parse times, less efficient execution plans and as the result, longer execution times.

Missing Joins in SQL

Missing joins between tables, scalar sub-queries, or EXISTS query blocks could cause severe performance impact on the SQL runtime, generate heavy logical and physical I/O, and affect overall database performance. The following example describes a simple case with a missing join:

```

1  SELECT
2      a.request_id,
3      c.party_id,
4      b.party_name
5  FROM
6      hz_parties a,
7      hz_parties b, -- Missing Joins in SQL (CHECK#24, SEQ#0)
8      hz_parties c
9  WHERE
10     c.party_id = a.party_id
11     AND c.party_number = :b1;

```

Here table HZ_PARTIES is not joined to any other table in the query, leading to cartesian join and row explosion in the query. Missing joins pattern could involve not only physical tables but also inline views (sub-queries) in FROM CLAUSES, as well as EXISTS expressions.

An effect to query's performance caused by missing join can be disastrous. When two tables have no join expression(s) between them the Optimizer has no choice but to create their CARTESIAN PRODUCT with the number of rows in such product being potentially extremely high. For example, for two 10,000-rows tables the result set would be 100,000,000 rows. Missing join may also indicate a functional bug - when working on a large and overly complicated body of SQL code, a developer may have overlooked the required additional join condition(s).

Note, that there may be CARTESIAN products, included into SQL on purpose. Additionally, the optimizer would correctly choose MERGE JOIN CARTESIAN when joining single row from table A with multiple rows from table B.

Missing Filters in SQL

It is very important to design BIP SQLs to use effective filters on the columns, that have the supporting indexes to ensure good performance and scalability for BIP reports with more users, more volumes, and higher concurrent workload. Consider an example of a SQL query that has no single filter expression:

```

SELECT
    c257413600,
    c241603489,
    c477250668,
    c150447593,
    c72705372,
    c354389364,
    c474672895
FROM
    (
        SELECT
            v112479773.invoice_distribution_id AS c257413600,
            ( 200 ) AS c241603489,
            ( 'AP_INVOICE_DISTRIBUTIONS' ) AS c477250668,
            v112479773.creation_date AS c150447593,
            v112479773.created_by AS c72705372,
            v112479773.last_update_date AS c354389364,
            v112479773.last_updated_by AS c474672895
        FROM
            ap_invoice_distributions_all v112479773
    ) t5912962;

```

Lack of filters in a query may cause all kinds of performance problems. When multiple tables are joined with no filters it may take long time to complete joins; it may also create noticeable pressure on Process Global Area (PGA) and temporary segment (TEMP) of a database. Make sure you always enforce EFFECTIVE DATE filters in your reports.

Too Many Column Expressions in GROUP BY Clause

When you design reports with aggregate functions, make sure you optimize the number of attributes in SELECT clause not to generate unnecessarily long GROUP BY clause. Having very long GROUP BY in a SQL may result in very expensive SORT and GROUP BY operations in the execution plan and as result, much higher CPU time to perform such operations.

Refer to a heavy GROUP BY example below:

```

720      xel5.importo_calcolato / xel5.measure_cnt_x_trx) <> 0 )
721      -- join payments history to calculate differences
722      AND xel5.source_trx_number = xph16.source_trx_number (+)
723      AND xel5.transaction_type = xph16.transaction_type (+)
724      AND xel5.plan_component_id = xph16.plan_component_id (+)
725      AND xel5.participant_id = xph16.credited_participant_id (+)
726
727      GROUP BY
728      xel5.earning_status,
729      xel5.tipo_commissione,
730      xel5.id_gara,
731      xel5.padre_gara,
732      xel5.codice_pos,
733      xel5.rag_sociale,
734      xel5.area,
735      xel5.canale,
736      xel5.process_date,
737      xel5.taglio,
738      xel5.piano_tariffario,
739      xel5.offering_new,
740      xel5.portfolio_id,
741      xel5.lv_lv,
742      xel5.reversal_family,
743      xel5.data_storno,
744      xel5.mobile_fisso,
745      xel5.importo_transazione,
746      xel5.transaction_type,
747      xel5.source_trx_number,
748      xel5.misura,
749      xel5.convergenza_fisso,
750      xel5.flag_tied,
751      xel5.flag_telefono_incluso,
752      xel5.tipo_offerta,
753      xel5.brand,
754      xel5.bundle_conv,
755      xel5.id_famiglia
  
```

Redundant Bridge Table

A bridge table is considered potentially redundant in SQL if it's (a), it joined but not selected from and (b), has same column used in more than one join condition. Refer to an example, using potentially redundant bridge table below:

```

SELECT
v196696337.item_number                AS c532986276,
organizationunittlpeo.name            AS c433637013,
v196696337.item_type                  AS c218999566,
v196696337.inventory_item_id          AS pka_inventoryitemid0,
v196696337.organization_id            AS pka_organizationid0,
organizationunittlpeo.organization_id AS pka_organizationunittlpeoorga0,
organizationunittlpeo.effective_start_date AS pka_organizationunittlpeoeffe0,
organizationunittlpeo.effective_end_date AS pka_organizationunittlpeoeffe1,
organizationunittlpeo.language        AS pka_organizationunittlpeolang0
FROM
egp_system_items_b_v v196696337,
hr_all_organization_units_f organizationunitpeo, -- Joined but not selected
hr_organization_units_f_tl organizationunittlpeo
WHERE v196696337.organization_id = organizationunitpeo.organization_id
AND organizationunitpeo.organization_id = organizationunittlpeo.organization_id
  
```

The query above does not select any data from `organizationunitpeo`. Additionally, the two joins below could be re-written to eliminate `organizationunitpeo` altogether:

```
-- original construct:
WHERE      v196696337.organization_id = organizationunitpeo.organization_id
AND organizationunitpeo.organization_id = organizationunittlpeo.organization_id

-- optimized construct:

v196696337.organization_id = organizationunittlpeo.organization_id
```

Each join processing requires IO and CPU time; the query run time increases more than linearly with the growing number of joins. If some joins are not required functionally, removing them will lower the query time.

Use LIKE in addition to SUBSTR Scenarios

Use of SUBSTR (and its variants, SUBSTRB, SUBSTRC, SUBSTR2, SUBSTR4) may be replaced or reinforced with additional LIKE conditions to facilitate index access paths if the filtered column is indexed:

- SUBSTR(column_name, 0/1, 0/N) and column_name LIKE 'XYZ%' should help to pick access path in case of LIKE condition.
- SUBSTR(column_name, N, 0/M) where N != 0/1 and column_name LIKE %PQR% → LIKE and SUBSTR behave similarly with slight changes in cost/row estimations.

SUBSTR on indexed columns with position=0/1 will have the effect of disabling index usage in access paths. Review the example with the suggested rewrites for SUBSTR to use additional LIKE below:

```
SELECT a.id
FROM
  substr_test_two a
WHERE substr(a.idm,1,3) = '300';

SELECT a.id
FROM
  substr_test_two a
WHERE substr(a.idm,1,3) = '300'
AND a.idm like '300%';
```

Leading Wildcard on LIKE Will Disallow Index Usage

LIKE condition may enable optimizer choose index access path on a corresponding column only if there is an ASCENDING index on the column and the WHERE clause uses the pattern column_name LIKE 'ABC%' with no leading wildcard %.

The following LIKE conditions never favor an index access path:

- LIKE '%ABC' / LIKE '%ABC%'
- Any LIKE condition if index is DESCENDING

In the following example SUBSTR_TEST table has a selective index on ID column and the applied filter is restrictive too. However, the leading wildcard on filter condition forces optimizer to use Full Table Scan:

```
SELECT * FROM SUBSTR_TEST WHERE ID LIKE '%300';
```

```
Plan hash value: 164547117
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		500	19000	147 (1)	00:00:01
* 1	TABLE ACCESS FULL	SUBSTR_TEST2	500	19000	147 (1)	00:00:01

```
Predicate Information (identified by operation id):
```

```
1 - filter("ID" LIKE '%300')
```

Carefully evaluate your LIKE conditions and the usage of ‘%’ wildcard in your BIP SQLs.

_TL Tables Should Have a Filter on Language Column

_TL tables are used in FA for storing records translated into all active languages enabled in the customer FA environment. Typically, when _TL tables are joined, their data needs to be restricted for locale using LANGUAGE filter. Not having userenv(‘LANG’) filter for every single _TL table could result in row explosion and subsequently slower SQL performance. Review the example of sub-optimal pattern below:

```
SELECT uci.effective_start_date
FROM   ff_user_tables_tl fut,
       ff_user_columns   fuc,
       ff_user_rows_f     fur,
       ff_user_rows_tl    furt,
       ff_user_column_instances_f uci
WHERE  fut.user_table_name LIKE 'PUBLIC_HOLIDAYS'
      AND fut.user_table_id = fuc.user_table_id
      AND fuc.base_user_column_name = 'HOLIDAYS'
      AND fut.user_table_id = fur.user_table_id
      AND fur.user_row_id = furt.user_row_id
      AND fuc.user_column_id = uci.user_column_id
      AND furt.user_row_id = uci.user_row_id
      AND uci.value IS NOT NULL

--Required additional LANG filters for this SQL are:

AND fut.language= userenv('LANG')
AND furt.language= userenv('LANG')
```

Sub-Queries Having Connect-by-Prior

Hierarchical connect-by-prior queries when used within sub-query blocks, for example EXISTS, IN clauses or correlated sub-queries, may cause severe performance issues. Typically, hierarchical queries result in the additional complexity due to their recursive nature. Such hierarchical queries, when incorporated within sub-queries may lead to egregious row explosions combined with sub-query un-nesting and other complex optimization techniques. Refer to the example of such sub-optimal usage below, with two correlated sub-queries in SELECT employing hierarchical queries:

```

SELECT ...,
  (SELECT substr(haou.name, 1, instr(haou.name, ' ', 1))
   FROM fusion.per_org_tree_node potn,
        fusion.hr_all_organization_units haou
   WHERE potn.tree_code = 'RMTest'
        AND haou.organization_id = potn.pk1_start_value
        AND LEVEL = 2
        START WITH potn.pk1_start_value = paam.organization_id
        CONNECT BY PRIOR potn.parent_pk1_value = potn.pk1_start_value
        AND PRIOR potn.tree_code = 'RMTest'
   ) "LOB 02",
  (SELECT haou.name
   FROM fusion.per_org_tree_node potn,
        fusion.hr_all_organization_units haou
   WHERE potn.tree_code = 'RMTest'
        AND haou.organization_id = potn.pk1_start_value
        AND LEVEL = 2
        START WITH potn.pk1_start_value = paam.organization_id
        CONNECT BY PRIOR potn.parent_pk1_value = potn.pk1_start_value
        AND PRIOR potn.tree_code = 'RMTest'
   ) "LOB 02 Description"
...

```

Materializing such query blocks or factoring them in a consolidated WITH clause are considered better performing options.

Outer Joins to _VL Views

Fusion _VL views typically select data from corresponding _B and _TL tables. If the report functionality requires Outer Join to _VL view, consider joining directly to _B or _TL tables, if the required data can be retrieved from the database tables. And, since Left Outer Join does not restrict rows from Left table, the _VL view can be replaced by _B base table or _TL table in the query instead.

Review the example below:

```

SELECT ...
  lookupvaluesmlspeo.lookup_category,
  lookupvaluesmlspeo.lookup_values_id
FROM moo_revn revenue,
     moo_opty opportunitiespeo
     zbs_lookup_values_vl lookupvaluesmlspeo
WHERE (revenue.opty_id = opportunitiespeo.opty_id
      AND revenue.status_code = lookupvaluesmlspeo.lookup_code(+)
      AND revenue.status_code_set_id = lookupvaluesmlspeo.set_id(+))
...

```

The more performant version using the base table _B is:

```

SELECT .....
  lookupvaluesmlspeo.lookup_category,
  lookupvaluesmlspeo.lookup_values_id
FROM moo_revn revenue,
     moo_opty opportunitiespeo,
     ZBS_LOOKUP_VALUES_B lookupvaluesmlspeo
WHERE (revenue.opty_id = opportunitiespeo.opty_id
      AND revenue.status_code = lookupvaluesmlspeo.lookup_code(+)
      AND revenue.status_code_set_id = lookupvaluesmlspeo.set_id(+))

```

If `_VL` is required by its functional design, then the SQL can be rewritten to use both LEFT OUTER JOIN to `_B` and LEFT OUTER JOIN to `_TL` and LANGUAGE filter for `_TL` table.

Order by on Multiple Table(s) Columns

If you design a report with multiple columns, make sure you review the final SQL, and carefully inspect its ORDER BY clause. Having very long ORDER BY list could result in very expensive SORT operation across multiple columns. If your report produces very large row count, the very last SORT operation could cause additional unnecessary overhead. Limit the number of attributes in ORDER BY clause to bare minimum.

Rewrite Aggregate Sub-Query in WHERE to Inline View in FROM Clause

SQL design with a table or view joined to an sub-query using aggregation over the same object could result in less optimal execution plan with extra steps accessing the same object more than once. The following example below illustrates the discussed sub-optimal design, with the view `hrt_review_periods_vl`, joined to itself in a sub-query retrieving the max (`end_date`) in WHERE clause. The more optimal SQL rewrite uses the equivalent functional logic using inline view in FROM clause:

Sub-optimal

```

1 SELECT
2   revprd.start_date,
3   revprd.end_date,
4   revprd.review_period_id,
5   hgpv.goal_plan_name gp_name,
6   hgpv.goal_plan_id
7 FROM
8   hrt_review_periods_vl revprd,
9   hrg_goal_plans_vl hgpv
10 WHERE
11   revprd.review_period_id = hgpv.review_period_id
12   AND revprd.status_code = 'A'
13   AND hgpv.goal_plan_active_flag = 'A'
14   AND revprd.end_date = (
15     SELECT
16       MAX(revprd2.end_date)
17     FROM
18       hrt_review_periods_vl revprd2
19     WHERE
20       revprd2.status_code = 'A'
21       AND trunc(revprd2.start_date) <= trunc(sysdate)
22   );
23

```

Optimal

```

1 SELECT
2   revprd.start_date,
3   revprd.end_date,
4   revprd.review_period_id,
5   hgpv.goal_plan_name gp_name,
6   hgpv.goal_plan_id
7 FROM
8   (
9     SELECT
10      revprd.start_date,
11      revprd.end_date,
12      MAX(revprd.end_date) OVER() max_end_date,
13      revprd.review_period_id
14     FROM
15      hrt_review_periods_vl revprd
16     WHERE
17      revprd.status_code = 'A'
18      AND trunc(revprd.start_date) <= trunc(sysdate)
19   ) revprd,
20   hrg_goal_plans_vl hgpv
21 WHERE
22   revprd.end_date = revprd.max_end_date
23   AND revprd.review_period_id = hgpv.review_period_id
24   AND hgpv.goal_plan_active_flag = 'A';

```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)
0	SELECT STATEMENT		1	146	11 (0)
1	NESTED LOOPS		1	146	7 (0)
2	NESTED LOOPS		1	146	7 (0)
3	NESTED LOOPS		1	81	4 (0)
4	NESTED LOOPS		1	53	4 (0)
5	TABLE ACCESS FULL	hrt_review_periods_b	1	34	3 (0)
6	SORT AGGREGATE		1	53	4 (0)
7	NESTED LOOPS SEMI		1	53	4 (0)
8	TABLE ACCESS FULL	hrt_review_periods_b	1	34	3 (0)
9	TABLE ACCESS BY INDEX ROWID	hrt_review_periods_tl	4	76	1 (0)
10	INDEX UNIQUE SCAN	hrt_review_periods_tl_pk	1	1	0 (0)
11	TABLE ACCESS BY INDEX ROWID	hrt_review_periods_tl	1	15	1 (0)
12	INDEX UNIQUE SCAN	hrt_review_periods_tl_pk	1	1	0 (0)
13	TABLE ACCESS BY INDEX ROWID BATCHED	hrg_goal_plans_b	1	28	2 (0)
14	INDEX RANGE SCAN	hrg_goal_plans_b_n4	3	1	1 (0)
15	INDEX UNIQUE SCAN	hrg_goal_plans_tl_pk	1	1	0 (0)
16	TABLE ACCESS BY INDEX ROWID	hrg_goal_plans_tl	1	65	1 (0)

Id	Operation	Name	Rows	Bytes	Cost (%CPU)
0	SELECT STATEMENT		1	133	7 (0)
1	NESTED LOOPS		1	133	7 (0)
2	NESTED LOOPS		1	133	7 (0)
3	NESTED LOOPS		1	60	4 (0)
4	VIEW		1	40	4 (0)
5	WINDOW BUFFER		1	53	4 (0)
6	NESTED LOOPS		1	53	4 (0)
7	NESTED LOOPS		1	53	4 (0)
8	TABLE ACCESS FULL	hrt_review_periods_b	1	34	3 (0)
9	INDEX UNIQUE SCAN	hrt_review_periods_tl_pk	1	1	0 (0)
10	TABLE ACCESS BY INDEX ROWID	hrt_review_periods_tl	1	15	1 (0)
11	TABLE ACCESS BY INDEX ROWID BATCHED	hrg_goal_plans_b	1	28	2 (0)
12	INDEX RANGE SCAN	hrg_goal_plans_b_n4	3	1	1 (0)
13	INDEX UNIQUE SCAN	hrg_goal_plans_tl_pk	1	1	0 (0)
14	TABLE ACCESS BY INDEX ROWID	hrg_goal_plans_tl	1	65	1 (0)

The original logic produces two scans of the same table, with the redundancy eliminated in the rewritten version on the right.

Redundant Attributes in SELECT Clause

Complex SQLs with multiple sub-queries could have attributes selected in the sub-queries, but never used propagated into enclosing parent Query Blocks for use in SELECT or WHERE clauses.

Eliminating redundant attributes not only makes your SQL cleaner but also reflects on the SQL performance:

- Optimizer may pick a better execution plan in some cases like an Index Full Scan instead of Table Full Scan if all useful columns are part of an index.
- Database will not waste precious memory resources for storing redundant columns/attributes across query blocks.
- A clean SQL text is easier to read and further optimize as needed.
- More complex patterns may have redundant attributes defined as correlated sub-queries in child SELECT clause, but never used in a parent, adding significant performance overhead for unnecessary sub-query processing.

Refer to the example of the sub-optimal SQL having a redundant attribute as a correlated sub-query:

```
SELECT
    t1c2,
    t2c1
FROM
    (
        SELECT
            t1.c1    t1c1,
            t1.c2    t1c2,
            t2.c1    t2c1,
            ( SELECT t3.c1
              FROM t3
              WHERE t3.c1 = t2.c1) t3c1    -- Redundant Attribute
        FROM
            t1,
            t2
        WHERE
            t1.id = t2.id
    )
WHERE
    t1c1 = 'ABC';
```

Deeply Nested Decode in SQL

Deeply nested DECODE statements in a SQL reflect over-complicated processing logic in BIP reports. There are several ways to address such complex deep nested DECODE statements, such as:

- Simplify the complex deep nested DECODE statements. The database optimizer would provide more accurate cardinality better cardinality estimations.
- Pre-calculate complex attributes and persist them to avoid runtime complex calculations.

```
SELECT
    lr.target_ledger_name  ledger_name_h,
    glb.ledger_id  ledger_id,
    decode(:p_period_type, 'PTD',
        decode(:p_currency_type, 'T', nvl(begin_balance_dr, 0) -
            nvl(begin_balance_cr, 0), 'S', nvl(begin_balance_dr, 0) -
            nvl(begin_balance_cr, 0), 'E',
            decode(glb.translated_flag, 'R', nvl(begin_balance_dr, 0) -
                nvl(begin_balance_cr, 0), nvl(begin_balance_dr_beq, 0) -
                nvl(begin_balance_cr_beq, 0)), 'C',
                nvl(begin_balance_dr_beq, 0) - nvl(begin_balance_cr_beq, 0)), 'D',
```

```
... decode(:p_currency_type, 'T', 0, 'S', 0, 'E', 0, 'C', 0),
```

The database optimizer may not be able to produce the accurate cardinality estimate and generate less optimal execution plan if such complex expressions are used filter/join condition. The performance of such SQL can be improved many folds by pre-calculating the complex deep nested DECODE attributes instead of computing them during runtime.

Attributes That May Force Late Filtering in Execution Plan

A filter on a view applied late in SQL execution plan, may disallow filter pushdown and lead to intermediate row explosions during the SQL runtime. Such later filters on views often cause excessive unnecessary processing, potentially suppressing optimal index scan plans or smart scan plans.

There are valid functional cases leading to late filtering such as:

- Hierarchical queries: inline views using hierarchical queries are always late filtered due to functional requirement.
- Derived columns: inline views filtered on derived columns have a functional requirement and a late filter is unavoidable.
- Aggregate queries: filters are pushed inside inline views based on aggregate queries, by optimizer, if the view is filtered on a group by column of the aggregate query.

There are few patterns, leading to late filtering, that may be possible to optimize for better performance.

1. ROWNUM pseudo column usage in inline view. If ROWNUM is not used in enclosing block's filter clause and there is another attribute from the same block, then ROWNUM is redundant and can be eliminated.

OPERATION	OBJECT_NAME	OPTIONS
SELECT STATEMENT		
VIEW		
Filter Predicates		
A.DEPTNO=20		
COUNT		
TABLE ACCESS	EMP	FULL

OPERATION	OBJECT_NAME	OPTIONS
SELECT STATEMENT		
VIEW		
Filter Predicates		
DEPTNO=20		
HASH JOIN		
Access Predicates		
ROWID=ROWID		
INDEX	DEPTNO_IDX	RANGE SCAN
Access Predicates		
DEPTNO=20		
INDEX	PK_EMP	FAST FULL SCAN

2. Inline views with analytic functions may be impacted by a subtle mistake of being late filtered on an unintended column which may cause unexpected filter pushdown issue. If there is another analytic function existing in same block and using common PARTITION BY Clause, it can help to push the filter in the inline view with analytic functions and deliver better performance:

```

select *
from (
  select empno eno,
         deptno as dno,
         avg(sal) over(partition by job, deptno) as avg_job_dept_sal,
         sum(sal) over(partition by mgr, deptno order by sal) sum_mgr_dept_sal,
         sum(sal) over() as sum_sal
  from emp
) ko
where ko.dno=34;

```

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			14	5
VIEW			14	5
Filter Predicates				
KO.DNO=34				
WINDOW		Sort	14	5
WINDOW		Sort	14	5
TABLE ACCESS	EMP	Full	14	3

```

select *
from (
  select empno eno,
         deptno as dno,
         avg(sal) over(partition by job, deptno) as avg_job_dept_sal,
         sum(sal) over(partition by mgr, deptno order by sal) sum_mgr_dept_sal,
         sum(sal) over(partition by deptno) as sum_sal
  from emp
) ko
where ko.dno=34;

```

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			1	4
VIEW			1	4
WINDOW		Sort	1	4
WINDOW		Sort	1	4
TABLE ACCESS	EMP	BY INDEX ROWID BATCHED	1	2
INDEX	EMPDEPTNO	RANGE SCAN	1	1

3. Pivot clause with late filtering can be optimized to push the filters before PIVOT clause. Refer to the example below:

```

--Sub-optimal pattern with the filters applied after PIVOT:

ORDER BY 3 ASC
) QUESTIONS PIVOT(MAX(ANSWER) FOR QUESTION_NUMBER IN ( 'Q01' , 'Q02' , 'Q03'
, 'Q04' , 'Q05' , 'Q06' , 'Q07' , 'Q08' , 'Q09' , 'Q10' , 'Q11' , 'Q12' , 'Q13' ) )
WHERE SUBJECT_ID = :SUBJECTID
AND PARTICIPANT_ID = :PARTICIPANTID

--Optimized pattern with the filters applied before PIVOT clause:

( . . .
AND HQLRV.SUBJECT_ID = :SUBJECTID
AND HQLRV.PARTICIPANT_ID= :PARTICIPANTID
. . .
) QUESTIONS PIVOT(MAX(ANSWER) FOR QUESTION_NUMBER IN ( 'Q01' , 'Q02' , 'Q03'
, 'Q04' , 'Q05' , 'Q06' , 'Q07' , 'Q08' , 'Q09' , 'Q10' , 'Q11' , 'Q12' , 'Q13' ) )

```

Positional Correlation in OR'ed EXISTS Clauses

When several EXISTS/IN clauses are using OR logic, it's important to keep all the external columns on the same side in join conditions. The database optimizer is syntax sensitive when it generate UNION-ALL operations in the execution plan.

The example below illustrates the impact of swapping sides for xla_ae_lines.ae_header_id and xla_ae_headers.ae_header_id in the join condition for the second EXISTS and producing more optimal explain plan:

```

SELECT
*
FROM
(
  SELECT
  *
  FROM
  xla_ae_lines l
  WHERE
  ( ( EXISTS (
    SELECT
    *
    FROM
    xla_ae_headers h
    WHERE
    h.ae_header_id = l.ae_header_id --> Different positional order
    AND h.ae_header_id = 1511989
  ) )
  OR ( EXISTS (
    SELECT
    *
    FROM
    xla_ae_headers h,
    dual
    WHERE
    l.ae_header_id = h.ae_header_id --> Different positional order
    AND h.ae_header_id = 1512025
  ) ) )
) v436914118;

```

Plan hash value: 403445960

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		2766K	593M	23417 (6)	00:00:01
1	FILTER					
2	TABLE ACCESS FULL	xla_ae_lines	2766K	593M	23410 (6)	00:00:01
3	UNION-ALL					
4	FILTER					
5	NESTED LOOPS		1	6	4 (0)	00:00:01
6	FAST DUAL		1	2	0 (0)	00:00:01
7	INDEX RANGE SCAN	XLA_AE_HEADERS_U1	1	6	2 (0)	00:00:01
8	FILTER					
9	INDEX RANGE SCAN	XLA_AE_HEADERS_U1	1	6	3 (0)	00:00:01

Predicate Information (identified by operation id):

```

1 - filter( EXISTS ( (SELECT 0 FROM "SYS"."DUAL" "DUAL", "xla_ae_headers" "H"
WHERE 1512025=<B1 AND "H"."AE_HEADER_ID"=<B2) UNION ALL (SELECT 0 FROM
"xla_ae_headers" "H" WHERE 1511989=<B3 AND "H"."AE_HEADER_ID"=<B4)))
4 - filter(1512025=<B1)
7 - access("H"."AE_HEADER_ID"=<B1)
8 - filter(1511989=<B1)
9 - access("H"."AE_HEADER_ID"=<B1)

```

```

SELECT
*
FROM
(
  SELECT
  *
  FROM
  xla_ae_lines l
  WHERE
  ( ( EXISTS (
    SELECT
    *
    FROM
    xla_ae_headers h
    WHERE
    h.ae_header_id = l.ae_header_id --> Same positional order
    AND h.ae_header_id = 1511989
  ) )
  OR ( EXISTS (
    SELECT
    *
    FROM
    xla_ae_headers h,
    dual
    WHERE
    h.ae_header_id = l.ae_header_id --> Same positional order
    AND h.ae_header_id = 1512025
  ) ) )
) v436914118;

```

Plan hash value: 136323277

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		4	952	12 (9)	00:00:01
1	NESTED LOOPS		4	952	12 (9)	00:00:01
2	NESTED LOOPS		4	952	12 (9)	00:00:01
3	VIEW	VW_SQL	2	26	7 (0)	00:00:01
4	HASH UNIQUE		1	12	7 (0)	00:00:01
5	UNION-ALL					
6	NESTED LOOPS		1	6	4 (0)	00:00:01
7	FAST DUAL		1	2	0 (0)	00:00:01
8	INDEX RANGE SCAN	XLA_AE_HEADERS_U1	1	6	2 (0)	00:00:01
9	INDEX RANGE SCAN	XLA_AE_HEADERS_U1	1	6	3 (0)	00:00:01
10	INDEX RANGE SCAN	XLA_AE_LINES_U1	4	1	2 (0)	00:00:01
11	TABLE ACCESS BY INDEX ROWID	xla_ae_lines	4	900	4 (0)	00:00:01

Predicate Information (identified by operation id):

```

8 - access("H"."AE_HEADER_ID"=1512025)
9 - access("H"."AE_HEADER_ID"=1511989)
10 - access("VW_SQL"="L"."AE_HEADER_ID")

```

Note

-- this is an adaptive plan

Use of Complex Fusion Database Views in BIP SQLs

Carefully review the logic of FA database views before using them in your BIP SQLs. Some database views, designed for specific functional requirements, could cause significant performance overhead if used in BI SQLs. Review some of the identified transactional views that should be avoided in BIP SQL data sets below.

- CN_SRP_PARTICIPANT_HDR_RO_V

The database view is designed to join three tables and uses VPD policies that could further complicate BI SQL performance. In the most cases using a single base table CN_SRP_PARTICIPANTS_ALL addresses the functional requirements and eliminates unnecessary redundancy, improving report performance.

- FUN_ALL_BUSINESS_UNITS_V
- HR_ALL_ORGANIZATION_UNITS
- HR_OPERATING_UNITS
- INV_ORGANIZATION_DEFINITIONS_V

These views often are picked for retrieving ORGANIZATION_ID and its NAME, brining in the additional complexity into BI SQLs. You can consider using HR_ORGANIZATION_UNITS_F_TL to retrieve ORGANIZATION_ID and NAME attributes:

```

SELECT
  businessunit.organization_id bu_id,
  businessunit.name bu_name
FROM hr_organization_units_f_tl businessunit
WHERE businessunit.language = userenv('LANG')

```

Alternatively, you may consider light database views FUN_BU_PERF_V or FUN_ORGANIZATIONS_V if your functional requirements cannot be met with HR_ORGANIZATION_UNITS_F_TL.

- PJF_PROJ_ALL_MEMBERS_V

Consider PJF_PROJ_TEAM_MEMBERS_V instead of heavier PJF_PROJ_ALL_MEMBERS_V view.

- INV_ORGANIZATION_DEFINITIONS_V

Check the feasibility of using the lightweight view RCS_ORG_DEFINITIONS_LW_V instead.

- EGP_SYSTEM_ITEMS_TL_V

CX customers, using Sales Products may consider using QSC_PRODUCTS_TL to retrieve the subset of products(items) that are sellable or serviceable, stored in EGP_SYSTEM_ITEMS.

- EGP_SYSTEM_ITEMS_VL and EGP_SYSTEM_ITEMS

Consider replacing these two DB objects with EGP_SYSTEM_ITEMS_BV.

Avoid Reusing OTBI Generated SQL in BIP Data Sets

OTBI generated database SQL text, full or partial, is strongly discouraged from reuse in any form in BIP Data Sets. The OTBI SQL text can be specific for admin and non-admin users, include dynamically generated security predicate to secure specific query blocks based on logical facts and dimensions involved, etc. OTBI report logic may involve more than one generated database SQL, with the SQLs run in OTBI execution context, set via its initialization blocks as well as various database features turned on or off for logical databases and their connection pools defined in OTBI abstract layer in RPD. A shortcut to 'borrow' OTBI SQLs, full or partial (security clauses, for example) rather than design carefully reports in BIP would lead to functional and often poor performance for such 'hybrid' BI reports.

Attributes with Mandatory UPPER Function in Filters/Joins

The following commonly used attributes have Function Based Index using UPPER function, so they should use UPPER in join and filter conditions:

```
UPPER(PER_PERSON_SECURED_LIST_V.PERSON_NUMBER)
UPPER(PER_ALL_PEOPLE_F.PERSON_NUMBER)
UPPER(FUN_ALL_BUSINESS_UNITS_V.BU_NAME)
UPPER(INV_ORG_PARAMETERS.ORGANIZATION_CODE)
UPPER(EGP_SYSTEM_ITEMS_B.ITEM_NUMBER)
UPPER(PER_ALL_ASSIGNMENTS_M.ASSIGNMENT_NUMBER)
```

Attributes with Mandatory TRUNC Function in Filters/Joins

The following commonly used attributes have Function Based Index using TRUNC function, so they should use TRUNC in join and filter conditions:

```
TRUNC(SVC_SERVICE_REQUESTS.OPEN_DATE)
TRUNC(SVC_SERVICE_REQUESTS.CLOSED_DATE)
```

Custom VPD Security in BIP SQLs

Unlike OTBI, which extracts and appends data security predicates to its database SQL, BI Publisher requires manual coding of data security clauses in its SQLs. The additional layers of security may come from VPD predicates, that get appended during SQL parse as part of its execution every time it's being run. The ability to customize VPD clauses provides the additional flexibility and offers extra security, however it does require very careful design and validation in all the cases where it would apply. Any shortcuts could lead to appending irrelevant security predicates via VPD, generating expensive Cartesian Merge Joins, row explosions, sub-optimal joins, and as the result significant impact on your BIP reports performance.

Use of Extensible Attributes in Filters and Joins

FA offers numerous extensible attributes (flexfields) for customizations across its Datamodel. Naturally, the flexfields can be used BIP reports to retrieve the mapped custom attributes. Remember that none of the extensible attributes have supporting indexes in FA tables, so any use of them in filters and joins has to be combined with attributes having indexes in the database. Use of flexfields as filters alone is not recommended.

Anonymous PLSQL Blocks in Factored WITH Sub-Query

Oracle Database rich functionality offers many features to address specific business requirements. One of such features is use of anonymous PLSQL blocks declared via factored WITH subquery in a SQL. Embedding PLSQL logic into a SQL, while very powerful feature, should be very carefully evaluated before implemented in BIP reports. In the overwhelming number of cases the same calculations can be handled in SQL directly, thus eliminating unnecessary complexity and delivering better performance.

Sub-optimal CONCATENATION in Execution Plans

Combination of OR and BETWEEN NVL construct may result in sub-optimal CONCATENATION choice in SQL execution plan, generated by the optimizer. Financial GL reports commonly use four tables "gl_je_headers", "gl_je_lines", "gl_ledgers", "gl_code_combinations" with the filters on ledgers (and/or periods) and segments in the example below:

```
AND (gl.name IN ( :p_ledger_name ) OR least(:p_ledger_name) IS NULL )
AND gcc.segment2 BETWEEN nvl(:p_from_account, gcc.segment2) AND
                        nvl(:p_to_account, gcc.segment2)
```

The combination of filters below (OR-based filter and BETWEEN NVL) results in miscalculated join cardinality and use CONCATENATION in the plan:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		62459	3964K	72152 (1)	00:00:03
1	CONCATENATION					
* 2	FILTER					
* 3	HASH JOIN		56781	3604K	58645 (1)	00:00:03
* 4	TABLE ACCESS FULL	gl_code_combinations	3731	41041	1168 (2)	00:00:01
* 5	HASH JOIN		1093K	56M	57462 (1)	00:00:03
* 6	HASH JOIN		20554	742K	1484 (1)	00:00:01
* 7	TABLE ACCESS FULL	gl_ledgers	34	918	14 (0)	00:00:01
8	TABLE ACCESS FULL	gl_je_headers	118K	1154K	1468 (1)	00:00:01
9	TABLE ACCESS FULL	gl_je_lines	6323K	102M	55896 (1)	00:00:03
* 10	FILTER					
* 11	HASH JOIN		5678	360K	13507 (1)	00:00:01
* 12	TABLE ACCESS FULL	gl_ledgers	34	918	14 (0)	00:00:01
* 13	HASH JOIN		32663	1212K	13492 (1)	00:00:01
14	NESTED LOOPS		32838	897K	12022 (1)	00:00:01
15	NESTED LOOPS		32838	897K	12022 (1)	00:00:01
16	TABLE ACCESS BY INDEX ROWID BATCHED	gl_code_combinations	373	4103	73 (0)	00:00:01
* 17	INDEX RANGE SCAN	ADXX_GL_CODE_COMBINATIONS_CAT	67		13 (0)	00:00:01
* 18	INDEX RANGE SCAN	GL_JE_LINES_N1	88		2 (0)	00:00:01
19	TABLE ACCESS BY INDEX ROWID	gl_je_lines	88	1496	32 (0)	00:00:01
20	TABLE ACCESS FULL	gl_je_headers	118K	1154K	1468 (1)	00:00:01

Optimizer manifests the same pattern if BETWEEN is replaced with the conditions below:

```
AND gcc.segment2 >= nvl(:p_from_account, gcc.segment2)
AND gcc.segment2 <= nvl(:p_to_account, gcc.segment2);
```

As a workaround, you can use /*+ NO_EXPAND */ hint to correct the query plan, or rewrite the logic to use IN-LIST bind variables:

```
--original pattern:

AND gcc.segment2 BETWEEN nvl(:p_from_account, gcc.segment2) AND
                        nvl(:p_to_account, gcc.segment2)

--optimized pattern:

AND (least(:p_segment2) IS NULL OR gcc.segment2 IN (:p_segment2));
```

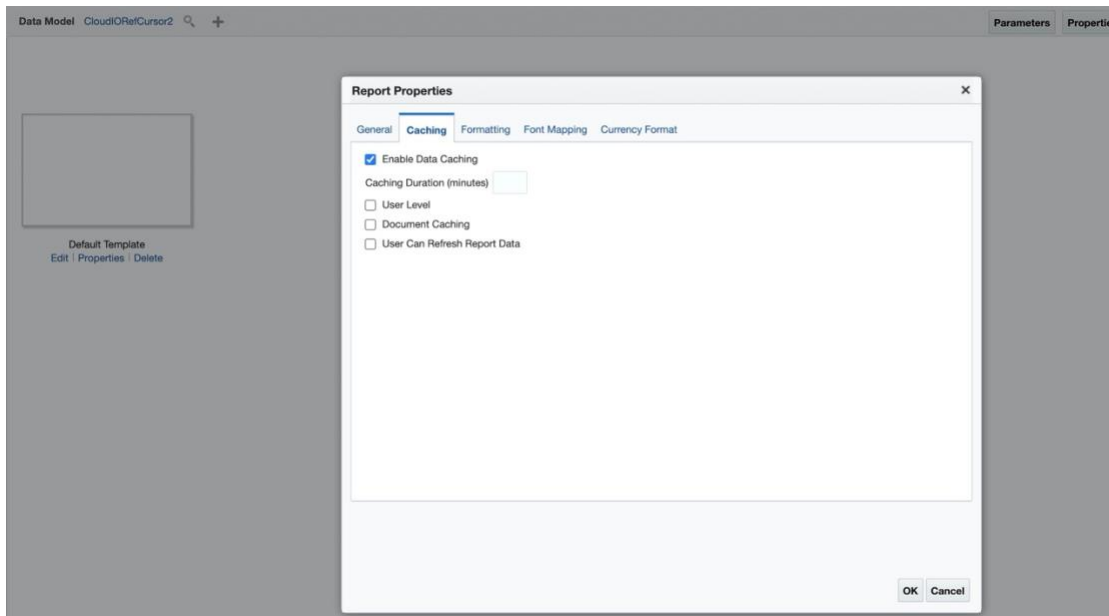
Fast Formula in BI Publisher Extract Reports

Use of fast formulas in BIP reports results in use of literals instead of bind variables in BIP reports, causing SQL hard parses every time reports get executed, and generating high number of 'row cache mutex' wait events if such reports get executed with high concurrency in FA database. The customers should consult product specific documentation, such as Payroll, for settings to turn on the use of bind parameters in BIP reports.

REPORTS AND DATA MODEL PROPERTIES

Data Caching for Online Reports

Enabling Data Caching for online reports can help to improve subsequent reports executions for the same report with the same parameters for the same user (if enabled at User level). The data would be retained in the cache for the specified duration (default 30 minutes). However, if you create BIP reports with Ref Cursor using **createReport()** API, it could result in generating a lot of cache entries in /tmp for high frequency, concurrent report executions, ultimately filling in all available space. Since createReport() API does not allow to uncheck Data Caching setting during report creation, you need to navigate to the report datamodel and manually uncheck it to prevent /tmp overflow with xmlp*tmp entries.



SQL Query Timeout

SQL Query Timeout property controls a SQL timeout for scheduled execution mode. The scheduled timeout limit is more relaxed as it does not involve any managed BI server threads.

Important! Oracle does not recommend changing this property and keep it enforced at the system level. Customized timeout values could result in heavy SQLs consuming more database resources and causing impact on the overall FA environment. Report developers should explore all the options to optimize their report SQLs and ensure them completing within enforced system limits.

BIP DATA GENERATION PERFORMANCE RECOMMENDATIONS

BIP Reports Template Size

Design your report templates to be keep their size small. The report size could grow with more templates added. If you schedule to run such complex template reports at high frequency, you could risk filling up all /tmp space with cached report definitions.

Refer to [BI Publisher Best Practices in FA SaaS document](#) for the detailed guidelines to design report templates.

Data Format and Data Generation Time

The choice of BIP report data generation format, PDF, Excel, RTF, or CSV can have performance impact on the overall report runtime. While PDF and Excel deliver rich formatting options, these formats take longer to generate, especially for very large data extracts. Review the example below, showing the additional four minutes spent in data generation time:

```
DataSet Name:report
  Number of elements:0
  Total Query Execution time (ms):0
DataSet Name:Parameters
  Number of elements:0
```

```

    Total Query Execution time (ms):0
DataSet Name:Triggers
    Number of elements:0
    Total Query Execution time (ms):1
DataSet Name:OutstEncRpt
    DataSet Type:sqlQuery-oracle
    Data Source Name:ApplicationDB_FSCM
    Is DataSet Execution Skipped:false
    DataSet DB fetch size:20
    DataSet Query Timeout:1800
    Total Number Of Times Query Executed:1
    Total Number Of Rows Processed:381
    Sql Action Id:Dp{1571445733}Sql{f9f69cbf}
    Sql Session
Details:SQLID:{fq9w2qb9x0bv9}SQL_HASH:{3553636201}SQL_EXEC_ID:{33554433}
    Number of elements:0
    Total Query Execution time (ms):282107
-----
Total Number of Datasets: 4
Total Number of Elements: 0
Total Parse Time (ms): 28
Total Datamodel Validation Time: .000 mseconds
Total Xplan Generation Time: .000 mseconds
Total Data Generation Time: 4.000 minutes
Total XML Data Size Generated: 186.61 KB
Processing Exceptions:

```

Consider using less expensive formats such as RTF or CSV for your data generation output, if they also satisfy the functional report requirements.

CLOB impact on Data Generation

CLOBs in BIP reports may cause additional overhead during report data generation phase. CLOBs inclusion into BIP reports could have significant performance impact. Such reports would not scale for large number of output rows, and not perform for high number of concurrent report instances. CLOBs usage should be limited to low frequency, low volume and low concurrency reports in FA environments.

If you observe long time spent in data generation for very few records returned by the executed data model, you need to check if you use any CLOB attributes in your BIP reports.

FUNCTIONAL AREA PERFORMANCE RECOMMENDATIONS

Financial BIP Reports

This chapter discusses some of the common performance patterns and recommendations, applicable to Financial BI Publisher reports in FA SaaS:

1. If you join "GL Lines" and "GL Balances" in Parent-Child relationship in your Data Model, consider merging them into a single Data Set. Such merged logic will not only help to avoid multiple expensive child data set executions, but also reduce the number of references to the same large FIN tables in a single report.
2. If you use a filter on `GL_JE_LINES.EFFECTIVE_DATE` (refer to the example below), make sure you include the additional filter on `PERIOD_NAME`, that is indexed in database. When end

users choose FROM/TO effective dates, PERIOD_NAME can be populated automatically and passed to the report to improve its performance:

```
AND GL_JE_LINES.EFFECTIVE_DATE >= to_date(:P_FROM_EFFECTIVE_DATE, 'YYYY/MM/DD')
AND GL_JE_LINES.EFFECTIVE_DATE <= to_date(:P_TO_EFFECTIVE_DATE, 'YYYY/MM/DD')
```

3. If you join GL_JE_HEADERS and GL_JE_LINES, make sure you use PERIOD_NAME for both lines & headers. The filter on GL_JE_HEADERS.PERIOD_NAME and joining to GL_JE_LINES does not guarantee the usage of the GL_JE_LINES_N1 index of GL_JE_LINES. A direct filter on GL_JE_LINES.PERIOD_NAME is functionally correct and very effective for performance as it has a supporting selective index in database.

```
SELECT ...
FROM
gl_je_headers gjh,
gl_je_lines gjl,
gl_code_combinations glcc
WHERE
gjh.je_header_id = gjl.je_header_id
...
AND gjh.period_name IN (:p_period_name)
AND gjl.period_name IN (:p_period_name)
```

4. Use of GL_SEG_VAL_HIER_RF and GL_SEG_VAL_HIER_CF tables: the _RF table is good for filtering by parent values, i.e., given a parent value, find all the children values and associated GL_CODE_COMBINATIONS. The _CF table is good for looking up the flattening hierarchy and find several parents at once, i.e., for a given GL_CODE_COMBINATION, find level 2, level 5, level 10 parents.
5. GL_SEG_VAL_HIER_CF filtered on 'TREE_VERSION_ID' only, may become a bottleneck in the execution plan, as it would get accessed via expensive index skip scan. Consider the following options:
 - a. Use the leading column(s) of indexes on TREE_STRUCTURE_CODE / ENTERPRISE_ID as filters without breaking functionality. Both these columns have only one single unique value combination, so including a filter below should force index range scan without breaking functionality:

```
AND HIER_CF.TREE_STRUCTURE_CODE='GL_ACCT_FLEX'
```

- b. The leading columns in indexes on TREE_STRUCTURE_CODE / ENTERPRISE_ID have only one unique value combination. If index range scan produce sub-optimal performance, you may try to use following hint to disable skip scan: /*+ OPT_PARAM('_optimizer_skip_scan_enabled','false') */ or force full table scan via the hint: /*+ FULL(GL_SEG_VAL_HIER_CF) */.
6. XLA_AE_HEADERS and XLA_AE_LINES tables:
 - a. Use the filter on (LEDGER_ID,APPLICATION_ID) to pick the available index XLA_AE_HEADERS(LEDGER_ID, APPLICATION_ID, ACCOUNTING_DATE) in your SQL.

- b. Add a direct filter on XLA_AE_LINES.APPLICATION_ID and XLA_AE_LINES.ANALYTICAL_BALANCE_FLAG if the latter is restrictive for your data.
- c. Use a filter on ACCOUNTING_DATE as PERIOD_NAME is not indexed:

```
AND (xla_ae_headers.accounting_date BETWEEN :p_acc_start_date AND
:p_acc_end_date)
AND (xla_ae_lines.accounting_date BETWEEN :p_acc_start_date AND :p_acc_end_date)
```

7. Avoid using database functions on XLA_EVENTS.TRANSACTION_DATE in filter or join predicates, as it has a supporting index on XLA_EVENTS(TRANSACTION_DATE) column.
8. Have additional filters on XLA_AE_LINES to get more accurate cardinality estimates if you rewrite the following constructs:

--original pattern:

```
AND ( nvl(xla_ae_lines.accounted_dr,0) - nvl(xla_ae_lines.accounted_cr,0) <> 0)
```

--optimized pattern:

```
AND ( nvl(xla_ae_lines.accounted_dr,0) - nvl(xla_ae_lines.accounted_cr,0) <> 0)
AND (xla_ae_lines.accounted_dr > 0 OR xla_ae_lines.accounted_cr > 0)
```

9. Consider removing ORDER BY from very large Financial extracts to eliminate expensive SORT operation.

BIP MEMORY GUARD SETTINGS IN FA SAAS

FA SaaS environments enforce BI Publisher memory guard properties at the system level. Refer to “BI Publisher Memory Guard Properties in Fusion Applications” (document [2370893.1](#)) describing the enforced memory guard settings in more details. For example, the following error message would be reported for scheduled reports hitting Memory Guard Data Size limit:

“oracle.xdo.memoryguard.XDODataSizeLimitException: Report data size exceeds the maximum limit.”

The memory guard settings have been carefully picked for FA SaaS environments to ensure their stability at the same time accommodate for the customer functional requirements to process and export large enough data volumes:

1. If you hit Memory Guard limiters in your report executions, consider reducing the report volumes by applying more restrictive filters to your reports.
2. CSV data generation format is the most relaxed in FA environments, so you may consider it for your data export instead of PDF or Excel.
3. If you run into report timeout error while executing it online, you can run it offline (i.e. scheduled) with more relaxed SQL timeout limit.
4. Any change to SQL timeout value in Datamodel for scheduled only reports is strongly discouraged, as it could have serious implications and result in significant impact on your FA environment performance.

BIP PERFORMANCE ERRORS: RECOMMENDATIONS AND WORKAROUNDS

This chapter covers most common errors related to BIP reports performance and potential solutions or workarounds if applicable. Remember to test carefully suggested workarounds before implementing in your Production environment.

ORA-10260: limit size (1048576) of the PGA heap set by event 10261 exceeded

The error message ‘ORA-10260: limit size (1048576) of the PGA heap set by event 10261 exceeded’ would be reported in FA SaaS environment to protect the database from ORA-04030 out-of-process memory error. It may come up for extremely large SQLs stuck in parsing for a long time due to the SQL sheer complexity. Another case could be misuse of stored PL/SQL or anonymous PLSQL block leading to memory leaks. The error message is a strong signal to work on optimizing your SQL and simplifying its logic.

ORA-01722: invalid number

ORA-01722 is often associated with data issues, when there is implicit or explicit conversion from VARCHAR2 to NUMBER data types. The error may pop with some dimensions having DEFAULT_VALUE column or hierarchies PK columns declared as VARCHAR2. Another potential source could be using numerous database views with sub-queries, that could lead to complex view merging and leading to the same error.

In the example below the two predicates are defined at the same level:

```
RatingLevelBPEO.RATING_MODEL_ID = ProfileTpScPrpBPEO.DEFAULT_VALUE  
UPPER(ProfileTpScPrpBPEO.COLUMN_NAME) = UPPER('RATING_MODEL_ID2')
```

If the first predicate gets executed before the second it will result in ORA-1722, as the DEFAULT_VALUE column with non-numeric values is joined to numeric column RATING_MODEL_ID. However, if the second predicate gets executed first, it will filter the records and complete successfully. To address the issue, apply the explicit conversion:

```
TO_CHAR(RatingLevelBPEO.RATING_MODEL_ID) = ProfileTpScPrpBPEO.DEFAULT_VALUE
```

You can try to suppress complex view merging by applying the hints or their combination below:

```
/*+ OPT_PARAM('_complex_view_merging','false') */  
/*+ OPT_PARAM('_optimizer_distinct_agg_transform','false') */
```

ORA-0600 Caused by Query Plan Exceeding 64K Plan Lines

Very complex SQLs with heavy redundant logic, joining multiple tables, querying complex database views, using long ORs and long IN-LISTS would take very long time to parse and if generated execution plan has more than 64,000 plan lines, fail with ORA-600 [xplSetRws:1]. If you run into this error, you must simplify your SQL logic, eliminate long IN-LISTS, replace complex DB views with fewer tables, etc.

ORA-01489: result of string concatenation is too long

SQL with LISTAGG in BIP reports may run into ‘ORA-01489: result of string concatenation is too long’, running into the SQL limit of 4000 bytes. Try to replace LISTAGG with XMLAGG:

```
SELECT trim(LISTAGG (to_clob( trans.name), ' | ') WITHIN GROUP (ORDER BY
trans.name)) QUALIFIER_NAME
FROM fusion.cn_rs_attributes_all_tl trans;
```

The rewritten SQL with XMLAGG complete successfully:

```
SELECT XMLAGG(XMLELEMENT(e,trans.name,' | ').EXTRACT('//text()') ORDER BY
trans.name).GetClobVal() QUALIFIER_NAME
FROM
  (SELECT DISTINCT trans.name
   FROM fusion.cn_rs_attributes_all_tl trans) trans
```

CONCLUSION

This document consolidates the best practices and recommendations for developing and optimizing performance for Oracle Business Intelligence Publisher for Fusion Applications Version 21B or higher. This list of areas for performance improvements is not complete. The document will be updated with more findings, revisions recommendations, so make sure you always use the latest version. If you observe any performance issues with your BIP reports, you should carefully benchmark any recommendations or solutions discussed in this article or other sources, before implementing the changes in the production environment.

CONNECT WITH US

Call +1.800.ORACLE1 or visit oracle.com.

Outside North America, find your local office at oracle.com/contact.



blogs.oracle.com



facebook.com/oracle



twitter.com/oracle

Copyright © 2021, Oracle and/or its affiliates. All rights reserved. This document is provided for information purposes only, and the contents hereof are subject to change without notice. This document is not warranted to be error-free, nor subject to any other warranties or conditions, whether expressed orally or implied in law, including implied warranties and conditions of merchantability or fitness for a particular purpose. We specifically disclaim any liability with respect to this document, and no contractual obligations are formed either directly or indirectly by this document. This document may not be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose, without our prior written permission.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Xeon are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Opteron, the AMD logo, and the AMD Opteron logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group. 2021

Oracle Business Intelligence Publisher Performance Recommendations for Fusion Cloud Applications

August 2021

Authors: Pavel Buynitsky, Oksana Stepaneeva, Eugene Perkov, Amar Batham

Contributing Authors: Surya Gupta, Wasimraja Abdulmajeeth, Rakesh Kumar

